Solving Large Maximum Covering Location Problems with a GRASP Heuristic

Case-study for stroke facility allocation in Vietnam

by
Fleur Theulen (2002050)

A thesis submitted in partial fulfillment of the requirements for the degree of Master of Science in Business Analytics and and Operations Research

Tilburg School of Economics and Management
Tilburg University

Supervised by:
Prof. Dr. Goos Kant (Tilburg University)
Prof. Dr. Ir. Dick den Hertog (Analytics for a Better World)
Prof. Dr. Joaquim Gromicho (Analytics for a Better World)

Date:
February 14, 2022

Abstract

In this paper a GRASP heuristic is implemented that is specifically designed for solving very large scale maximum covering location problems. The main goal is to allocate new stroke treatment centers in Vietnam, which is a huge problem of 350,000 potential facility locations and over 300,000 population points. The novelty of this thesis is twofold: the size of the Vietnam case vastly exceeds the size of any other test instance solved in the literature, and we are the first to convert the fastest swap-based local search procedure in the literature from a p-median to a maximum covering setting. The GRASP heuristic first constructs a randomized greedy solution, then performs a restricted first improvement or a best improvement local search, and finishes with back- and forward path-relinking as an intensification strategy. The whole heuristic is based on a sparse matrix implementation, to overcome memory issues. The Vietnamese stroke facility allocation problem is solved on 3 detail levels, for various values of the service distance and number of new facilities. The heuristic is tuned and tested on the middle detail level, for which the optimal values can still be computed, and finds high quality solutions with a maximum deviation of 0.025% from the optimum. Solutions for the highest detail level are found in less than 2 hours, and each solution improves on the global optimum of the middle detail level. Coverage increases of up to 1.4% are observed, which is equivalent to the optimum with 5 more facilities on the middle detail case. This demonstrates the added value of being able to solve very large maximum covering problems.

Acknowledgements

I would like to express my gratitude for the guidance, ideas and enthusiasm of my supervisors. Special thanks to Dick den Hertog for coming up with this interesting research topic and for our inspiring weekly meetings, during which we discussed solutions for my thesis as well as for many other social issues. I would like to thank Joaquim Gromicho for our fruitful collaboration, his relentless enthusiasm, and for his brilliant insights in increasing code efficiency. I am also grateful for the constructive feedback and striking questions from Goos Kant, which elevated the level of this thesis.

This research would not have been possible without the network of Analytics for a Better World, and the input of the World Bank. Together, these institutions are able to put the outcomes of this research into use, thereby contributing to the progress of developing countries.

Finally, I definitely want to thank my parents and my dear friends. Mom and dad, thank you for always listening to me, for having such a welcoming home waiting for me, and for your advice, unsolicited or not. My student years would have been incomplete if I had not shared the countless hours in the library, the awful cups of coffee, and the many laughs with my friends, who I will definitely see in many more years to come.

Contents

1 Introduction **4**

2 Literature Review **6**
2.1 Facility location problems . . . 6
2.2 History of solving large MCLPs heuristically . . . 6
2.3 Large scale $p-$median heuristics . . . 9
2.4 Selected approach . . . 10

3 Theoretical background **11**
3.1 Maximum Covering Location Problem . . . 11
3.2 Construction Phase . . . 12
3.3 Local Search Phase . . . 13
3.4 Path-relinking . . . 14

4 Data **17**
4.1 Essential data sources . . . 17
4.2 Data pre-processing . . . 17
4.3 Data exploration . . . 18

5 Implementation **20**
5.1 Basic elements . . . 20
5.2 Sparse matrix implementation . . . 21
5.3 Construction phase . . . 24
5.3.1 Greedy approach . . . 24
5.3.2 Combine randomness and greediness . . . 27
5.4 First improvement local search . . . 28
5.5 Best improvement local search . . . 31
5.5.1 Constructing *loss*, *gain*, and *extra* . . . 32
5.5.2 Solving discrepancy between indices and facility IDs . . . 33
5.5.3 Updating *loss*, *gain* and *extra* . . . 34
5.6 Path-Relinking . . . 38
5.6.1 Path-relinking with the advanced approach . . . 38
5.7 The full algorithm . . . 40

6 Results Vietnam **41**
6.1 Analysis of the separate phases . . . 41
6.1.1 Construction phase . . . 41
6.1.2 Local Search Phase . . . 43
6.1.3 Path-Relinking . . . 44
6.2 Analysis of the whole process . . . 46
6.3 Outcomes of the tuned algorithm on the largest case . . . 50

7 Performance compared to the literature **54**

8 Conclusion **56**

References **60**

A Appendices **61**

1 Introduction

Every year, around 200,000 citizens of Vietnam suffer from a stroke according to the Ministry of Medicine of Vietnam [29]. This stroke is fatal for half of these patients, and for 90% of the patients that do survive a stroke, it results in neurological or physical problems, such as cognitive disorders, reduced mobility and problems with eating and speaking. With strokes being responsible for 22% of total deaths in Vietnam, it was the leading cause of death according to the Global Burden of Disease study conducted in 2019 [7]. Rymer et al. [27] show that the sooner a patient is treated after a stroke, the lower the probability of death or disability after a stroke. At the moment there are only 80 health facilities in Vietnam which are well equipped to perform the specific treatments needed after a stroke. Because Vietnam is a large country in which the existing stroke facilities are unequally distributed, many people have to travel too far before they can get help.

The Vietnamese government wants to invest in new health centers in order to improve the availability of stroke care. The aim of this paper is to develop a method that can advise the government on where to locate these new facilities. Our objective is to maximize the number of people who can access a facility within some fixed distance. In mathematical terms, this is called a maximum covering problem. We do not take a maximum capacity into account for the number of people that can be treated in each facility, making it an uncapacitated maximum covering problem. The fixed distance is called the covering or service distance, which is the maximum distance considered to be acceptable to travel to a stroke facility. People living within this service distance from a facility, are counted as covered. The locations of the new facilities are chosen from a predefined list of potential facility locations, and population data is needed in order to determine where the demand for stroke facilities occurs. Road network data is utilized to calculate the actual travel distance between any two points, which results in more reliable outcomes than using euclidean distances.

What the service distance should be is not immediately clear, because Armijo-Olivo et al. [1] explain that there are different time windows for a stroke treatment to be effective, depending on the kind of treatment. In addition to the service distance, the number of new facilities that can be located is also an important parameter. The government has not decided on an available budget yet, so this number of new facilities is also not predefined. To be able to give a proper advice, we will solve the problem for various values of the service distance and of the number of new facilities, such that decision makers can assess the effect on the percentage of people covered and decide on the parameters.

This research is conducted in collaboration with the World Bank Group and the Analytics for a Better World (ABW) initiative. The World Bank Group is a partnership between five financial institutions, and together they strive to end extreme poverty and increase shared prosperity. These goals are pursued by investing in sustainable development projects, sharing knowledge and giving advice in developing countries. On the other hand, ABW facilitates research with positive societal impact, by providing guidance to students and establishing valuable connections with the business.

Together, the World Bank and ABW have already executed a similar study on which this research will progress: locating new hospitals in a small country named Timor-Leste. This was also a maximum covering problem, but on a much smaller scale. This project has already moved from the design to the implementation phase, during which the World Bank is working together with the local government to put the outcome of this research to practice.

In the research about Timor-Leste, it was observed that the time and computer memory needed to find the optimal solution increase when the list of potential facilities grows, when more data points are used to represent the population, or when the service distance increases. The largest case that was tested for Timor-Leste, consisted of 15,000 potential locations, almost 200,000 population points and used a service distance of 5 kilometers. For the Vietnam case, this will increase to more than 350,000 potential locations, 300,000 population points and service distances up to 100 km. At this scale, we receive memory errors when attempting to solve the problem optimally, and even if we could work a way around the memory issues, we expect a long solving time. In this paper we therefore develop a heuristic, instead of solving the problem exactly. This heuristic does not need to find the optimal solution, but it should be able to handle very large problems such as the Vietnam case. The goal is to develop a method that can find a close-to-optimal solution in a relatively short time, without encountering memory issues on an average workstation.

The most important requirements for the heuristic are clear: it has to be time and memory efficient, and still find high quality solution for very large maximal covering problems. In addition to these requirements, we want a solution that can be implemented in the Geospatial Planning and Budgeting Platform. This is the tool that is used by the World Bank to visualize the results of the optimization, which gives a governing party more insight in the results. Furthermore, it would be valuable if there would be a 'quick and dirty' option, which can quickly give an indication of the number of people that can be covered for a given service distance and number of new facilities. This allows a governing party to roughly assess the effect of changes in their budget or acceptable travel distance without waiting for a long time, and evaluate whether their expectations are realistic. Using such a first analysis, a few parameter values can be chosen for which the heuristic gets more running time to find high quality solutions.

As a last requirement, we would like the heuristic to be generalizable, which means that the same technique could be used to solve similar problems for other, possibly even bigger use cases. There already is an impending project for locating farmer's markets in India, which also has the maximal covering problem setting. In fact, many of the United Nations Sustainable Development Goals are formulated in a maximal covering way, so if the heuristic is generalizable, it can have many future applications. This would increase the societal impact of this thesis substantially. To this end, we focus on using basic techniques that have shown to be useful in various problem settings, are easy to maintain, and are not too dependent on parameter tuning.

The structure of this thesis is as follows. In Section 2 we review the related literature, in Section 3 we describe the theoretical background needed to understand the problem and solving approach. In Section 4, all data related topics are discussed. In Section 5 there is an extensive explanation about how we implemented the heuristic. In Section 6 we analyse the results obtained for the Vietnam case. We apply the developed heuristic on a case from the literature in Section 7, so we can test if the performance is generalizable. Finally in Section 8, we draw and discuss our conclusions, and give several recommendations for further research.

2 Literature Review

2.1 Facility location problems

There are several kinds of facility location problems, each alternative formulation serves a slightly different goal. Daskin [6] distinguishes three main categories: covering problems, median problems and others. Covering problems use an all-or-nothing approach for the demand, according to which a demand point is counted as covered if it lies within the service distance of one the facilities. Median problems aim to minimize the average distance between the demand points and their appointed facility, where the average distance is weighted by the respective demand. The residual category contains location problems that do not have a clear common component. An example of a problem that falls in this category is the p-dispersion model, which maximizes the minimum distance between any two facilities.

Median problems are often used in the commercial sector, because by minimizing the average distance, the total costs are also at their lowest. A p-dispersion formulation could be useful for facilities that do not want to steal clients away from other facilities, for example shops of the same franchise. Finally, covering problems are more suitable for non-profit purposes such as placing hospitals, schools or voting offices, because you want to make these facilities accessible to all people. Since covering all demand is often very expensive, Church and Revelle [4] developed the Maximum Covering Location Problem (MCLP) in 1974. Instead of covering all demand, the coverage is maximized with a predefined number of facilities. This formulation gives a more realistic approach, because now you can calculate an optimal solution for any given budget.

Garey and Johnson [10] proved in 1979 that the MCLP is non-deterministic polynomial-time hard, or NP-hard in short, like many other facility location problems. For NP-hard problems, the scientific consensus is that there exists no polynomial time algorithm that can solve them, although this has never been proven. These problems are thus considered to be 'hard' to solve. The size of NP-hard problems for which the optimal solution can be found by exact solvers grows over time, as technological advances allow computers to perform calculations faster and the underlying algorithmic methods improve. In addition to the research towards exact solution methods, many heuristics have also been developed. Heuristics do not guarantee to find an optimal solution, but generally they are more efficient and scale better than exact methods, making it possible to handle larger problems. We will now discuss some important heuristics that have been developed to solve the maximal covering problem.

2.2 History of solving large MCLPs heuristically

Table 1 shows an overview of many of the heuristics that have been applied to the standard MCLP problem in the literature so far. We only included papers in this overview if their approach to solving a MCLP was novel, or if the maximum size of the tested instances was larger than other articles reported so far. We focus on algorithms solving the basic uncapacitated maximal covering problem, without extensions.

Author(s)	Solution method	Max instance*
Galvão and ReVelle (1996) [9]	Lagrangian heuristic	150 × 150
Resende (1998) [18]	Greedy Randomized Adaptive Search Procedures (GRASP)	10,000 × 1,000
Galvão et al. (2000) [8]	Comparison Lagrangian and surrogate relaxations	900 × 900
Lorena and Pereira (2002) [14]	Lagrangian and surrogate heuristic using Hillsman's edition	900 × 900
Pereira et al. (2007) [17]	Column generation adjusted with Langragian	800 × 800
ReVelle et al. (2008) [24]	Heuristic concentration	900 × 900
Senne et al. (2010) [28]	Decomposition heuristic	3,000 x 3,000
Zarandi et al. (2011) [32]	Genetic algorithm	2,500 × 2,500
Rodriguez et al. (2012) [26]	Population-based iterated greedy	3,000 × 3,000
Calderin et al. (2017) [3]	Algorithm portfolio	2,500 × 200
Máximo et al. (2017) [16]	Intelligent guided GRASP	8,000 × 8,000
Máximo et al. (2019) [15]	Intelligent guided GRASP with path-relinking	8,000 × 8,000
Cordeau et al. (2019) [5]	Benders decomposition	40,000,000 × 100

*largest instance solved by the heuristic in paper: # demand points × # potential facility locations

Table 1: The main solution methods proposed for the MCLP

In 1996, Galvão and Revelle [9] developed a heuristic that uses the Lagrangian Relaxation (LR) of the maximal covering problem. They relax the covering constraint, resulting in a problem that can be solved easily for a fixed set of Lagrange multipliers. The covering constraint normally ensures that a demand point is only counted as covered if one of the potential facility locations covering this demand point is chosen. After relaxing this constraint, the heuristic starts with any set of initial Lagrange multipliers and solves the relaxed problem. This solution is then used to find a feasible solution for the original problem, and an upper and lower bound for the objective. The bounds and solution are improved iteratively, by using subgradient optimization to find better Lagrangian multipliers, and using them to solve the relaxed problem again. Finally, the algorithm is complemented with an interchange heuristic, which is applied to each new solution. This technique was tested on an instance of 150 demand points and 150 potential locations. It also produces an upper bound, so the maximum gap between the optimum and the best found objective can be calculated.

The next development in this field that we want to highlight was in 1998, when Resende [18] published a paper in which he applied a Greedy Randomized Adaptive Search Procedure (GRASP) to the MCLP. Each GRASP iteration consists of constructing a solution with an adaptive greedy approach and improving this solution using local search techniques. In the construction phase, the greedy function calculates for each facility the additional number of demand points covered if that facility is chosen. Sorted according to this greedy metric, the first facility is chosen randomly from the top $\alpha\%$ of the potential facilities. Subsequently the greedy metric is updated, taking into account the demand already covered by the chosen facility. This process is repeated until p facilities are chosen. This construction phase is followed by a local search phase, which continues until the solution is the best solution in its neighbourhood, and thus a local optimum. For the time in which this paper was published, the GRASP method could already handle large instances, of 10,000 customers and 1,000 potential locations.

In 2000, Galvão et al. [8] compared heuristics based on the Lagrangian relaxation and the surrogate relaxation, and found there was no significant difference between these two methods in solving MCLP. In the surrogate relaxation, the relaxed constraints are not added to the objective as is done in the Lagrangian relaxation, but the covering constraints of all demand points are aggregated into a single knapsack constraint. Two years after this paper,

Lorena and Pereira [14] progressed on this topic, and constructed a hybrid Lagrangian/-surrogate relaxation which outperformed both heuristics separately. Both Galvão et al. and Lorena and Pereira tested their heuristic on a maximum size of 900 by 900, the largest test instances found in the literature up to that point.

Note that in the paper of Lorena and Pereira, the MCLP was reformulated as a P-median Problem (PMP) in order to be able to apply the hybrid heuristic. This reformulation was first presented by Hillsman in 1984 [11], in a paper in which Hillsman shows that there are several location problems that can be reduced to a PMP. By choosing the cost coefficients of a PMP in a certain way, optimizing this p-median problem is equivalent to optimizing the maximum covering problem. This allows for using solution methods that are actually designed for solving a PMP, to solve a MCLP.

Pereira et al. [17] published a paper in 2007, in which they also formulated the MCLP as a PMP, and tried to solve it using column generation. According to their paper, this reformulation lead to instances that were difficult to solve with column generation, due to high oscillations in the dual solutions. In an effort to reduce this problem, they multiplied the dual solutions with a Lagrangian/surrogate multiplier, which is a known stabilization method. This multiplier slowly increases as new, better columns are generated, reaching the value of one when the solution reaches optimality. The stabilization was successful, however all in all the heuristic did not improve on previous solution methods.

Revelle et al. [24] applied heuristic concentration to the maximum covering problem, but did not solve larger instances than the years before. In 2010, Senne et al. [28] developed a method that managed to solve a test instance of 3,000 customers by 3,000 potential facilities. They implemented a decomposition approach, that essentially splits the facility location problem into several smaller scale problems. How these splits are made is based on cluster partitioning. The constraints that connect the sub-problems are relaxed, and exact solving methods can be used to solve the small problems. Combining the solution results in an upper bound for the original problem, which is tighter than the Lagrangian linear relaxation.

In the years that followed, several more papers were published that applied heuristics to solve the MCLP, all with similar test instance sizes. Zarandi et al. [32] created a genetic algorithm in 2011, followed a year later by Rodriguez et al. [26], who developed a population-based iterated greedy approach. Then in 2017, Calderin et al. [3] constructed an algorithm portfolio. These papers all had some interesting ideas, but did not manage to solve larger instances or construct better upper bounds, so we do not discuss these papers in depth.

The article that managed to take a decent step forward in the size of the location problem, was written by Máximo et al. in 2017 [16]. Their method is based on the article of Resende from 1998 [18], which describes the application of GRASP on a MCLP for the first time. The difference lies in that Máximo et al. manage to address the weakest property of GRASP, which is the independence between iterations. By constructing a solution and improving it with a local search, information is gathered about the value of potential locations. In GRASP, this information is thrown away after each iteration, so the iterations are independent. By introducing dependence between iterations, the gathered information can be used in subsequent iterations. This article uses an artificial neural network which

influences the solution constructed in the next iteration. Every solution found at the end of the local search phase, is used to build and improve the neural network, so the quality of the solutions in the construction phase will improve as the algorithm progresses. The neural network functions like an intelligent adaptive memory, which enables the algorithm to learn from past iterations. Máximo et al. named this heuristic Intelligent-Guided Adaptive Search (IGAS), and observed that it found the same or better solutions in less time than GRASP on all of their test cases.

Two years later, Máximo et al. [15] published another research in which they extended both IGAS and GRASP with path-relinking. Path-relinking is the exploration of paths connecting two local optima, and is usually executed at the end of each GRASP iteration. It is a common method to introduce dependence between iterations, but it works differently from the adaptive memory in IGAS. Path-relinking is used to improve the solution after the local search, whereas the memory in IGAS influences the construction phase. After extending the heuristics, they were tested on the same test instances as in the previous paper, and path-relinking appeared to be a valuable addition to both. Its added value was larger on GRASP was larger than on IGAS, because in GRASP there was no dependence at all before the path-relinking was implemented, while IGAS already used the neural network memory to incorporate knowledge from earlier iterations.

The last article that we briefly discuss, was published by Cordeau et al. [5] in 2019. They managed to solve instances with a huge number of demand points, up to 40 million, using an algorithm based on Benders decomposition. However, they remark that this only works on instances for which the number of potential facility locations is very small in comparison to the number of demand points. As this will not be the case in our application, we will not be able to use this technique.

We observe that the instances on which the MCLP heuristics have been tested in the literature so far, are not very large compared to the size of the problem that we are aiming to solve. To the best of our knowledge, the largest case in the literature is 8,000 demand points by 8,000 potential facility locations, whereas the Vietnam case is of size 400,000 by 350,000. However, there are several papers that solve the $p-$median problem for instances of 90,000 by 90,000 nodes, which is already a lot closer to our desired size. As we have seen in the papers of Lorena and Pereira [14] and Pereira et al. [17], we can use heuristics developed for the PMP to solve a MCLP, if we first reformulate the problem. Therefore we will now describe two studies in which large scale PMP instances were solved.

2.3 Large scale $p-$median heuristics

The first article that we discuss, was written in 2012 by Avella et al. [2], and combines Lagrangian relaxation with column generation. First, they relax the $p-$median problem in a similar way as done by Galvão and Revelle [9] for the MCLP, and optimize the Lagrange multipliers using subgradient optimization. The resulting solution is used to identify which facility locations are promising, and a restricted problem is constructed with these locations. Using a commercial MIP solver, the restricted problem can be solved fairly quickly. Because the solution of the subgradient optimization and of the restricted problem influence each other, this process is repeated several times. The article manages to solve very large instances of 90,000 nodes, and also produces a lower bound for the optimum. Note that a lower bound for a PMP corresponds to an upper bound in the MCLP. In order to

be able to solve such large instances, a solution needs to be found to the memory problem, because storing the whole distance matrix takes up too much memory on a normal workstation. This problem is addressed by incorporating column generation in the heuristic, which allows them to only store the parts of the distance matrix that correspond to variables with negative reduced costs.

A different promising technique was applied to the same large scale PMP instances by Irawan et al. [12] in 2014. They use a Variable Neighbourhood Search (VNS) with several phases, and manage to solve the instances faster and with more precision than Avella et al. However, this method uses spatial aggregation, which aggregates demand points which are located in the same square of a certain grid into a so-called Aggregate Spatial Unit. In order to do this, you need the x and y coordinates of all demand points, but due to the translation from MCLP to PMP, you cannot express the location of the demand points in x and y coordinates. Thus we cannot use this technique on the reformulation of MCLP.

Even though the sizes of the solved cases are promising, there is an important counter-argument. By rewriting a MCLP to a PMP, you increase the number of constraints and variables. In a maximal covering problem, the number of constraints and variables is a linear function of the problem size, while in a $p-$median problem, both the constraints and variables grow quadratically. The MCLP formulation is thus expected to scale better, and suffer less from memory limitations.

2.4 Selected approach

We conclude that, to the best of our knowledge, no heuristic has ever been applied to a maximum covering problem of the magnitude we envision, so there is not one clear direction to go in. We decide that we will not use the reformulation of a MCLP to a PMP because of the increase in constraints and variables, thus we do not consider methods using this translation. In the remaining papers, we look for heuristics that are promising in the instance size that they have solved. If the heuristic managed to solve problems at a larger scale than other heuristics could at that time, we consider it a promising heuristic. Furthermore, we prefer relatively simple methods. This will allow us to focus on implementing the heuristic in a very efficient way. A smart implementation can yield great reductions in time and memory usage, even if the theoretical worst-case complexity is unfavourable.

From the literature review, we observe that two papers managed to outperform the existing literature of that time in terms of instance size, both of them used heuristics based on GRASP [16, 18]. Moreover, the MCLP instances in the most recent one, written by Máximo et al. [16], are the largest problems solved in the literature so far. Máximo et al. [15] also conducted a follow-up research, in which they tested the path-relinking extension often used in GRASP. The results show that path-relinking improved both their basic implementation of GRASP, and the more sophisticated IGAS heuristic.

We conclude that the GRASP based heuristics seem to be the most promising direction for our application. In the next section we will therefore analyse the steps of the GRASP methods, describe the choices that can be made, and which extensions are often used. We opt to focus on the simple GRASP instead of the IGAS heuristic, which will allow us to concentrate our effort on implementing it efficiently.

3 Theoretical background

In this section, we will provide all the mathematical background needed to understand our problem and the techniques that we will use to solve it. We will first formally define the Maximum Covering Location Problem (MCLP), then we will discuss the steps of GRASP in the subsequent sections. In addition to the construction and the local search phase, we will also look into path-relinking, because Máximo et al. [15] observed this was a valuable extension of GRASP when applied to a large MCLP.

3.1 Maximum Covering Location Problem

The MCLP can be formulated in a multitude of ways, the following is the original formulation of the problem, as proposed by Church and Revelle [4]:

$$\text{Maximize} \sum_{i \in N} a_i x_i \tag{1}$$

$$\text{Subject to} \sum_{f \in C_i} y_f \geq x_i \qquad \forall i \in \mathcal{N} \tag{2}$$

$$\sum_{f \in \mathcal{M}} y_f = p \tag{3}$$

$$x_i \in \{0, 1\} \qquad \forall i \in \mathcal{N} \tag{4}$$

$$y_f \in \{0, 1\} \qquad \forall f \in \mathcal{M} \tag{5}$$

Where:

$\mathcal{N} = \{1, ..., n\}$ is the set of all demand points

$\mathcal{M} = \{1, ..., m\}$ is the set of all potential facility locations

$x_i =$ binary decision variable which attains the value one if point i is covered by one of the chosen facility locations

$y_f =$ binary decision variable which attains the value one if location f is chosen to accommodate a facility

$p =$ number of facilities to be located

$a_i =$ volume of demand at point i

$d_{if} =$ distance between demand point i and facility location f

$S =$ service distance; maximum distance within which a facility covers a demand point

$C_i = \{f : f \in \mathcal{M}, d_{if} \leq S\}$ is the set of facility locations that can cover demand point i

Maximizing the objective function of a MCLP results in a solution that maximizes the total covered demand. Demand is considered to be covered if its corresponding demand point lies within the cover or service distance S of at least one facility. In order to define this model, two binary decision variables are needed. Decision variable y indicates for each location whether or not a facility will be located there, and x indicates for each demand point whether or not it is covered by one of the facilities. Note that the value of x is a direct consequence of y, as at least one of the locations that lie within the service distance of a certain demand point i must have value one in y, otherwise i is not covered and x_i will get the value zero. For every point i, C_i keeps track of which locations f satisfy $d_{if} \leq S$, and are thus able to cover i. The relationship between x and y is ensured by the constraints (2). Constraints (3) restrict the number of facilities that can be chosen

to a maximum of p. Finally, Constraints (4) and (5) are the binary constraints for x and y.

In our application of the MCLP, the demand points i represent small clusters of inhabitants, and the demand a_i is the number inhabitants in those clusters. From here on we will refer to demand points with either demand or population points. The facilities that we are considering are stroke treatment centers. Furthermore, we will solve the MCLP for various values of p and S, in order to provide insight into the effect of their decisions to the decision-makers. When deciding on the budget, and thus on the maximum number of facilities, they should take the trade-off into account that exists between p and the maximum number of people that you can cover. A higher p means that more people can be covered, or a smaller service distance can be sustained. However, the marginal effect decreases as p increases, meaning that for each extra facility the additional covered demand decreases. Solving the problem for different values of S and p provides insight into this trade-off, such that policy makers can make well-informed decisions.

3.2 Construction Phase

As explained in the literature review, GRASP consists of multiple iterations, where each iteration consists of a construction and a local search phase. There are numerous methods that can be used in the construction phase to build up the solution, the most common ones are discussed by Resende and Werneck [22] and by Resende and Ribeiro [20]. A good construction method is efficient and yields high quality solutions, but also ensures there is enough variation between solutions in different iterations. If the quality is too low, the local search phase will take a lot of time, so fewer executions can be finished within a certain time limit. If the variation is too little, the local search can frequently arrive at already visited local optima, essentially wasting iterations that could have been used to explore new parts of the solution space. We will now highlight a few of techniques commonly used in the construction phase:

- *random*: p facilities are selected randomly from all m potential facility locations. The selection process is very efficient, with complexity $O(m)$. Checking if a customer is within service range of a facility costs $O(pn)$, therefore the total complexity is $O(m + pn)$.
- *randomized greedy (rgreedy)*: In a standard GRASP implementation, the randomized greedy method is most often used. As described in Section 2.2, the first facility is chosen uniform randomly from the top $\alpha\%$ of potential facilities, which are ranked according to their greedy value. Here, the greedy metric measures the additional demand covered by a facility if that facility is chosen. The top $\alpha\%$ of facilities form the Restricted Candidate List, or RCL. Once the first facility is chosen, all remaining facilities are reevaluated with the greedy metric, and the next facility is selected in the same way. This process is repeated until p facilities are chosen. Every evaluation of the greedy metric has complexity $O(mn)$, so the whole selection process has complexity $O(pmn)$.
- *random plus greedy (rpg)*: Randomness and greediness are also combined in this approach, but now the first $\alpha * p$ facilities are chosen at random, and the rest of the facilities are selected in a greedy way. This procedure takes $O((m + \alpha pn) + (1 - \alpha)(pmn))$ time.
- *proportional greedy (pgreedy)*: This method is similar to the *rgreedy*, but instead of a uniform random choice, another underlying distribution is used with a bias

towards choosing a facility with a higher rank. Let $r(f)$ denote the rank of facility f according to the greedy metric. Then a linear bias chooses facility f with a bias $bias(r) = 1/r$. Instead of using a linear bias, it is also possible to use a log bias, $bias(r) = log^{-1}(r + 1)$, an exponential bias, $bias = e^{-r}$, or a polynomial bias of order n, $bias(r) = r^{-n}$. The probability that a facility f is chosen is equal to its bias divided by the sum of the biases of all facilities in RCL. The complexity of this algorithm is the same as in rgreedy, $O(pmn)$.

- *sample greedy (sample)*: This approach is closely related with the randomized greedy approach, but has been adapted to improve the efficiency. In every iteration, a random sample of $q < m$ potential facilities is considered and the best facility according to the greedy metric is chosen. q has to be small enough in order to improve efficiency and to make sure the solutions vary enough between iterations. The complexity of the algorithm is $O(m + pqn)$.

The different methods vary in terms of average solution quality, variation and efficiency. *rgreedy* is the default method in GRASP, its solution quality and variation depends on the value of α. A value close to 1 corresponds to high variation but low solution quality, and vice versa for α close to 0. Together with *pgreedy*, it belongs to the least efficient methods according to their worst-case complexity $O(pmn)$. As described above, these three methods each have their own way to balance solution quality and variability, which can be tuned to specific applications.

The construction of *random* solutions is the most efficient, but it is known that the solution quality is very low. *sample* and *rpg* both try to find the middle ground between the aforementioned methods, being more efficient than *rgreedy* and *pgreedy*, and yielding solutions of higher quality than *random*. There is not one method that is always the best to use, this depends on the kind of problem, its size, the available solving time, and on the implementation of the construction method. This last factor, the implementation, can have a big impact on the worst-case complexity and on the actual time needed to find the randomized greedy solution. In section 5.3, we will describe in detail which steps we have taken to increase the efficiency of these construction methods. As there is not a single preferred method, we will test several options to see which one yields the best results for our application.

3.3 Local Search Phase

In the local search phase, the solution found in the construction phase is improved by investigating its neighbourhood, and adapting the solution if a better solution is found. This process continues until you arrive at a local optimum, so there are no solutions in the neighbourhood that yield a better objective value than the last found solution itself. The time that it takes to find a local optimum and the quality of such a solution is determined by several factors. The main factors are the strategy of the local search, the quality of the starting solution, the definition of a neighbourhood, and the technique used to evaluate solutions in the neighbourhood. We will discuss the meaning and influence of these factors in this section.

The strategy of a local search procedure determines when a move to a new, better solution is made. We distinguish first improvement local search, during which a move is made as soon as a better solution is found, from best improvement local search, during which the

whole neighbourhood is investigated before the best move is executed. According to the research of Resende and Ribeiro [21], there are many applications in which the first improvement strategy finds the same local optimum as the best improvement strategy, but it finds them a lot faster.

As shown by Resende in 1998 [18], low quality solutions need more time to get to a local optimum than higher quality solution, because more swaps need to be done and processed before the local optimum is reached. If a lot of time is spent in the local search phase then fewer GRASP iterations can be completed within a certain time limit. Moreover, low quality solutions often end up in local optima with a lower objective value. These two effects explain why it is preferable to spend some more time in the construction phase to find a higher quality solution, which ultimately results in a big increase in the local search speed.

The definition of the neighbourhood of a solution also plays a large role in the efficiency of a local search, a smaller neighbourhood will likely be more efficient, but could also yield lower quality local optima. Within GRASP, it is common practice to define the neighbourhood as all the solutions that differ from the considered solution by one facility. The best solution in this neighbourhood is called the 1-swap local optimum, because it can be found by 'swapping' one active facility with one inactive facility. Teitz and Bart were the first ones to develop a local search technique based on swapping facilities in 1968 [30]. Their basic procedure was a time-consuming process however, especially for larger problems. Fortunately, more efficient implementations were developed that can accelerate the search.

These more efficient implementations use different techniques to evaluate solutions in the neighbourhood. In 1983, Whitaker [31] introduced a method that finds the facility that can best leave the solution, for a given inactive facility entering the solution. In 2007, Resende and Werneck [23] enhanced this method such that all possible swaps could be considered simultaneously. That is, it determines which facility should leave the solution and which inactive facility should enter in order to get the best improvement in objective value. The method uses three auxiliary structures, which use extra memory, but make sure the same calculations do not have to be repeated as the search continues. They conducted several experiments in which random starting solutions were improved with a best improvement local search, and managed to finish the local search up to 800 times faster than Whitaker's implementation. The paper describes this approach in a p-median problem setting, of which the conversion to a maximum covering setting is not trivial. In section 5.5 we describe how the approach of Resende and Werneck works, and how we adjusted it to make it suitable for a maximum covering problem.

3.4 Path-relinking

As we have explained in the literature review, the independence between iterations is a big disadvantage of GRASP. In 1999, Laguna and Martí [13] first proposed to apply path-relinking to the solutions found in each GRASP iteration, so not all knowledge obtained in previous iterations was lost. The general idea of path-relinking is to explore a path connecting two promising solutions, in order to investigate if an even better solution can be found on this connecting path. The idea behind this technique is that many good solutions occur in each others vicinity in the solution space, and therefore a better solution possibly can be found on the path connecting two already high quality solutions. Path-relinking thus focuses on researching a promising part of the solution area more intensively, which

means that it is an intensification strategy.

Promising solutions are collected in a so-called pool of elite solutions. In each GRASP iteration, after the completion of the local search phase, a solution is randomly selected from the pool of elite solutions. This solution will serve as the *guiding solution*, and the solution found in the local search phase will be the *initial solution*. By swapping active and non-active facilities, the initial solution is transformed into the guiding solution, and for every intermediary solution the objective is calculated. The best solution found on this path is selected, and forms the final result of this GRASP iteration with path-relinking. This solution will also be considered as a candidate for the pool of elite solutions. If it is allowed to enter the pool, it could be randomly selected in one of the iterations that follow, in that case it would serve as the guiding solution itself.

The pool of elite solutions is the most valuable if it contains high quality as well as diverse solutions. We want the elite solutions to be sufficiently different from each other, in order to prevent that very similar paths are investigated many times. In order to achieve this, a candidate will only enter the pool of elite solutions if it satisfies on of the following decision rules:

1. The maximum capacity of the pool is not reached yet.
2. The candidate solution is the best solution found so far, it will replace the solution in the pool with the lowest objective value.
3. The candidate solution is sufficiently different from any of the existing elite solutions, and it has a better objective value than the elite solution with the worst objective value. The candidate solution will replace its most similar solution in the subset of elite solutions that are of no better quality than the candidate.
4. The candidate solution is not sufficiently different from at least one of the existing elite solutions, but its objective value is better than one of these similar solutions. The candidate solution can enter the pool in the place of the worst similar solution.

Resende and Ribeiro [20] specified the first three of these rules as the standard rules which are used in GRASP for selecting candidate solutions. The third rule is specifically designed to ensure a certain amount of diversity in the pool. The fourth rule was formulated by ourselves, because we think it is valuable to include a candidate solution if it is better than its similar solutions in the pool. By replacing one of these similar solutions, we expect that this does not influence the diversity of the pool.

Once the pool contains at least one elite solution, path-relinking can be executed after the local search phase. The guiding solution is selected at random from the pool of elite solutions, either following a uniform distribution or a distribution biased towards the higher quality solutions in the pool. The transformation from initial to guiding solution is realised by swapping facilities which only occur in the guiding solution with facilities that only occur in the initial solution. At each step, all possible remaining swaps are considered, and the one yielding the best solution is executed. This process is repeated until the initial solution is identical to the guiding solution.

So far we have discussed the standard path-relinking approach, but there are also some variations that are likely to obtain even better results or to increase efficiency. We will

discuss the difference between forward, backward, back- and forward, mixed, and truncated path-relinking. Forward is the standard approach, where the GRASP solution is the initial solution, and an elite solution is the guiding solution. In backward path-relinking, these roles are switched. The result of this switch is that the neighbourhood around the elite solution is explored more thoroughly, whereas in forward path-relinking the search focuses more on the neighbourhood around the GRASP solution. As the elite solutions are usually of better quality, it is expected to find better solutions near to the elite solutions. In the experiments conducted by Resende and Ribeiro [19] in 2003, backward path-relinking indeed led to better results compared to the standard approach.

The same experiments show that a combination of backward and forward path-relinking works even better, but it also costs double the time to explore both these trajectories. Therefore Ribeiro and Rosseti [25] suggested a mixed path-relinking strategy. In this strategy, one step is taken from the initial towards the guiding solution, followed by a step from the guiding towards the updated initial solution. This way, two paths are generated from both solutions towards the other one, which meet somewhere in the middle. The created path is again a path linking the original initial and guiding solution. This mixed strategy explores the neighbourhoods of both the initial and the guiding solution thoroughly. In the experiments of Ribeiro and Rosseti, the mixed strategy took approximately the same time as the forward strategy, and managed to find the best solutions out of all strategies.

4 Data

In this section we discuss which data we have used in order to solve the facility location problems, which pre-processing steps we have applied to the data to obtain the necessary information, and perform a brief data exploration. In essence, there are only two things we need to get from the data: the number of people belonging to every demand point and an overview of which facilities can cover which demand points. The first part can be obtained directly from the data, to get the second part, we need to combine several data sources. This process will be explained in this section.

4.1 Essential data sources

All the data that we have used are available from open-source platforms. Even though the World Bank considers the utilized sources to be the most reliable among the freely available data sources, we should note that the data quality is not guaranteed. We have used the following sources:

- Population data: we use Facebook data at a detail level of one squared kilometer. The data indicates how many people live in every squared kilometer, and contains the corresponding longitude and latitude of the center of such a population grid.
- Road network data: we use the road network data of Open Street Map. This data is likely to be incomplete, especially for the rural areas in less developed countries. Before decisions are actually made using the heuristic developed in this paper, the local government should be consulted in order to see if this open source data can be complemented with infrastructure data from the government.
- Potential facility data: we want to be able to place a new facility anywhere in the country, so we divide the country into a grid, in which each square represents a possible location. For the Vietnam case, we use three different instances, considering 10 km, 5 km and 1 km for the length of one square in the grid. Once the final solution is computed, policy makers can choose the best sites to locate the stroke facilities within the advised squared areas.

The goal of this heuristic is to solve the facility problem at the 1km scale for the Vietnam case. We use the smaller scale facility data to simplify the process of developing the code, and to test the performance of our heuristic against the optimal solution. The optimal solution can still be computed at the 10 km and 5 km scale, so these results and computing times can serve as a benchmark. We assume that if the heuristic finds results close to the optima of these smaller instances, it will also find solutions close to the optima at the 1 km level, although we cannot compute these optima to check this. For each grid size, we will test three values for the service distance: $S = 20$, $S = 50$ and $S = 100$, which means that we have 9 cases in total.

4.2 Data pre-processing

Before we are able to use these data sources to develop a heuristic, we need to pre-process the data. The process of pre-processing the data is fairly in-depth, and out of scope for this thesis. Therefore we will only briefly discuss what steps have been taken to translate the raw data into useful data structures.

The population data does not need much pre-processing, we only need to remove the population points which are already covered by the existing stroke facilities. The data is grouped per squared kilometer, so we consider the center of this square as the demand point, and the number of inhabitants in that area as the corresponding demand. The potential facility data is similar, for every location the data contains an ID and its corresponding coordinates. The construction of the grid data can actually be considered to be part of the data pre-processing. The shape of a country is taken as input, this is polygon data which has coordinates as its vertices. This area is then divided into squares of a fixed size, and the coordinates of the center of each square are saved as the location of the potential facilities.

Finally, we need to use the road network data to determine which facility can cover which population point. The road network data is processed using the Pandana package in Python. This package is able to efficiently calculate the shortest route in a network from one point to another. Many coordinates of population points and facility locations lie slightly off the network, so auxiliary edges are added to the network to connect these locations. These auxiliary edges connect a data point in a straight line to its closest point in the network, measured using the haversine distance. Next, we calculate the haversine distance between all facilities and population points, and select the facility-population pairs for which this distance is smaller than the set service distance S. Finding the shortest path in a network is much more expensive than calculating the haversine distance, and clearly the network distance is always at least as long as the haversine distance, so filtering on the haversine distance first saves a lot of time. Using Pandana, the road distance is calculated for the remaining facility-population pairs, and the pairs that are actually within S kilometers from each other are selected. We will refer to these pairs as the feasible combinations.

4.3 Data exploration

Table 2 displays several characteristics of the data. The number of potential facilities, indicated by $|\mathcal{M}|$, is determined by the size of each grid. The service distance S determines which percentage of the population is already counted as covered by the current stroke facilities in Vietnam. Naturally, the proportion of covered people increases when a larger service distance is used. This is also visible in the number of population points $|\mathcal{N}|$ that remain after removing the ones that are already covered. Take $S = 100$ for example, then $|\mathcal{N}|$ is less than half of the original 406,784 population points.

In the last column we report the sparsity of each case. The sparsity is the percentage of all facility-population pairs that are feasible combinations. As we will see in the implementation description in chapter 5, the sparsity of a case has a big influence on the time and memory needed to solve that problem.

grid-size	$\|\mathcal{M}\|$	S	$\|\mathcal{N}\|$	Covered	Sparsity
10 km	3,555	20	381,161	38.20%	0.09%
5 km	14,235	50	302,013	67.55%	0.53%
1 km	356,101	100	191,128	83.73%	1.91%

Table 2: Data characteristics of Vietnam cases

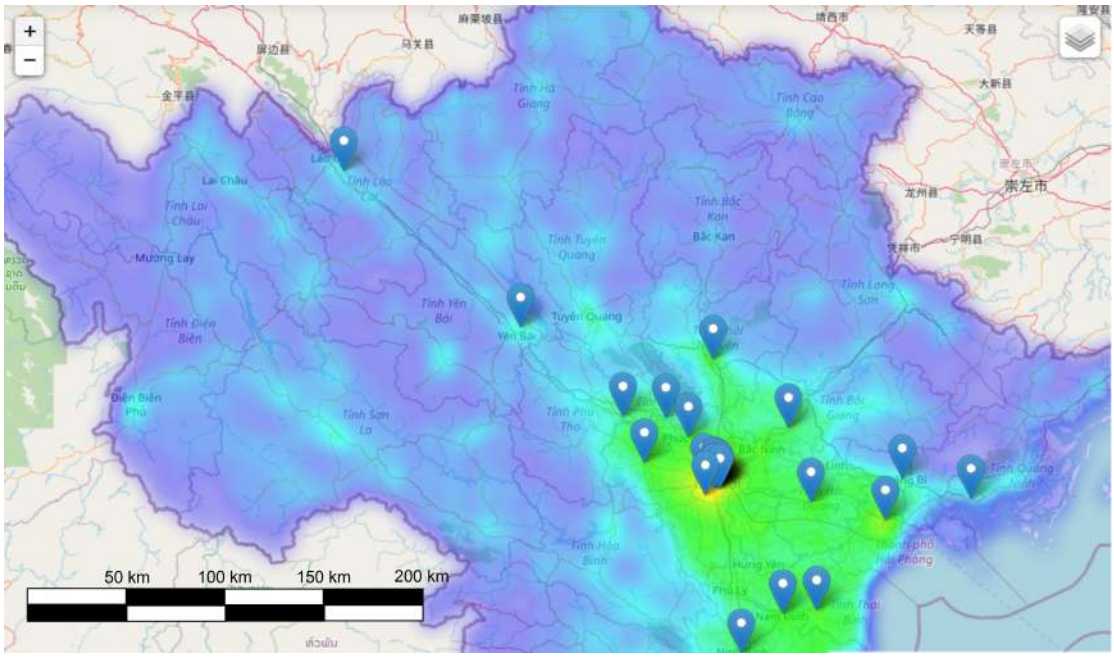


Figure 2: Existing stroke facilities in the north of Vietnam

In order to get a feeling for the current situation, we have plotted the existing stroke facilities on the map of Vietnam. Figure 2 shows the north of Vietnam, the blue markers are the locations of the current stroke facilities, the scale bar at the bottom left shows the scale of the map, and the heat map reflects the population density of each area. From this image it is immediately clear that there is a large inequality between rural and urban areas in the accessibility of facilities. In the rural areas there are very few facilities, whereas the cluster of markers in the capital city Hanoi indicates there are 11 facilities in that city alone. Given that many more people live in the big cities, it makes sense that most of the facilities are located there, but in the current situation many people living in rural areas will have to travel very far to get a treatment after suffering a stroke, increasing the risk of dying or permanent neurological damage.

For the sake of completeness, we have included a similar image showing the distribution of existing stroke facilities in the rest of Vietnam in Figure 10 in the appendix. Again, we see an uneven distribution, with most stroke facilities concentrated in the area of Ho Chi Minh City, the biggest city of South Vietnam. A difference with the image on this page, is that there are also some green, more urban areas, which are located far away from the nearest facility. Summarizing, we can conclude from these images that there is plenty of room for improvement in the accessibility of stroke care.

5 Implementation

The main goal of our heuristic is to be able to solve very large scale problems, so an efficient implementation is essential. The implementation needs to be efficient in both time and memory, as both can be limiting factors as the problem grows. In this section we will first discuss what structures we use to represent the binary decision variables, the demand vector and the feasible combinations. Next, we will dive deep into the implementation of each of the phases of GRASP. We start by describing how to efficiently construct the greedy solution and its randomized variations. Then, for the local search phase, we explain our implementations of both the first and the best improvement local search strategy. Finally, we discuss how this best improvement strategy can be turned into a path-relinking algorithm. We conclude this section with an algorithm that shows the connection between all the separate techniques. Note that the whole implementation chapter is intended for the interested reader, as the material is fairly in-depth.

5.1 Basic elements

In section 3.1 the mathematical formulation of a maximum covering location problem is given. There are several ways in which you can represent the parameters and variables of this model. These choices influence the efficiency of calculating the objective corresponding to a solution, and the amount of memory needed to execute our heuristic. In this section we will show how we decided to represent the problem.

We start by defining the set of potential facilities $\mathcal{M}$ and the set of population points $\mathcal{N}$. The sets are filled respectively with facility IDs $0, \ldots, m-1$ and population point IDs $0, \ldots, n-1$, which are used to uniquely identify each point, and function as indices at the same time. We will illustrate the whole process described in this implementation chapter using a small example. $\mathcal{M}$ and $\mathcal{N}$ of this example are given below:

$$\mathcal{M} = \overbrace{\begin{pmatrix} 0 & 1 & 2 & 3 & 4 \end{pmatrix}}^{m} \qquad \mathcal{N} = \overbrace{\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \end{pmatrix}}^{n}$$

Next, we have the population vector *pop*, which holds the number of inhabitants belonging to each population point. *pop* takes the place of the demand vector a in the MCLP model we defined in section 3.1.

$$pop = \overbrace{\begin{pmatrix} 20 & 10 & 5 & 10 & 3 & 30 & 5 & 8 \end{pmatrix}}^{n}$$

In the previous section, we described that we pre-process the data in order to find the feasible combinations, which are the facility-population pairs that are within the service distance from each other. This information is saved in a binary matrix B, with dimensions $m \times n$. The rows represent the facilities, the columns the population points, and each 1 indicates a feasible facility-population pair. The binary matrix for our small example is shown below.

$$B = \overbrace{\begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}}^{n} \Big\} m$$

The decision variables of the MCLP model are x and y, both binary vectors. y indicates whether or not a facility location is chosen, and x specifies if a population point is covered, which is a direct consequence from the choice for y. In a heuristic approach, many solutions are evaluated in order to find the best one. This means that solution vector y is input, but we need to create the corresponding x vector ourselves. This is done as follows: select the p rows of B corresponding to the ones in y, and take the sum over the rows of this sub-matrix. This results in a vector of length n, which we call the coverage vector *cov*. The coverage vector specifies how many facilities cover each population point in the current solution. A population point is covered if at least one active facility is covering it, so the formula for decision variable x is $x = \mathbb{1}_{\{it_{cov} \geq 1\}}$. Again, we illustrate this process with our example:

$$y = \text{ξ} \left\{ \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \right. \begin{array}{l} \rightarrow cov = \overbrace{\begin{pmatrix} 1 & 1 & 0 & 2 & 0 & 1 & 1 \end{pmatrix}}^{n} \\ \rightarrow x = \mathbb{1}_{\{cov \geq 1\}} = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 1 & 1 \end{pmatrix} \end{array}$$

Now we have all the ingredients to calculate the objective *obj* corresponding to a given solution, while adhering to all constraints of the MCLP model. The objective is simply the dot product of x and the population vector *pop*, which is equal to the total number of covered people.

$$obj = \mathbb{1}_{\{cov \geq 1\}} * pop = 20 + 10 + 10 + 5 + 8 = 53$$

In our heuristic, many different solutions will have to be evaluated in an attempt to find a high quality solution. In the construction, local search and path-relinking phase, the new solution that is considered always differs by only one facility from the previous solution. So instead of calculating the objective from scratch for every new solution, it is beneficial to have a way of efficiently computing the objective change caused by a small modification in the solution. The coverage vector *cov* allows us to do the latter. If one active facility is swapped with an inactive facility, you can simply take the coverage vector of the old solution, subtract the row of B corresponding to the leaving facility and add the row of the entering facility. The resulting vector is the coverage vector of the new solution, the same vector as you would obtain by summing over the p rows of B corresponding to the new solution. Adding and subtracting a row of B has complexity $O(2n)$, or $O(n)$, as constants are usually not considered in the complexity analysis. Summing over the p rows of B on the other hand, has complexity $O(pn)$. As we will test instances up to $p = 200$, using the coverage vector is much more efficient.

Once we have updated the coverage vector, we compute x by checking for each entry in *cov* whether it is larger or equal to one, and calculate the new objective by taking the dot product of x and *pop*. Both operations have complexity $O(n)$, so the total complexity of calculating an objective change is $O(n)$.

5.2 Sparse matrix implementation

Although the binary matrix is a clear and intuitive way to represent which facilities can cover which population points, it has one large disadvantage. The full binary matrix would have $350,000 \times 300,000 = 1.05\text{e}11$ entries if it were constructed for the Vietnam case with

grid-size = 1 and $S = 50$. Even though a binary matrix only needs 1 byte per entry, this matrix would still take up approximately 98 Gigabytes. This is much bigger than the average workstation can handle, so we would either have to invest in a virtual machine with more memory, or come up with a smarter alternative for the matrix B. In section 2 we reported the sparsity of each case, which varied between 0.09% and 1.91%, depending on the service distance S. These low percentages imply that converting B to a sparse matrix can result in a large reduction of memory usage. In this section we explain which type of sparse matrix we have implemented, and how the conversion to a sparse matrix implementation can improve the efficiency of the objective calculation.

A sparse matrix only remembers the nonzero values of a dense matrix. To be more precise, it contains the coordinates of a nonzero value, and its corresponding value. There are several ways to work with sparse matrices in Python, the SciPy library contains 7 different packages that each construct the matrix in a different way. Each technique has its own stronger and weaker properties, some are faster in constructing or adapting the matrix, and others are better with arithmetic operations. However, they are all designed to represent matrices in which the nonzero entries can be any value, whereas in our case, all nonzero entries in B are equal to one. This means it is sufficient to only remember the locations of the nonzero values, not the values themselves.

In order to utilize this advantage, we opted not to use any of these standard packages, but to work with two dictionaries, which function like sparse matrices. A dictionary is a data structure that stores multiple keys and their corresponding values, so if you give a key as input, it returns the value belonging to that key. The first dictionary, *row*, represents the rows of the binary matrix. The potential facility IDs are the keys of the dictionary, and each value is a vector containing all population points that can be covered by the corresponding facility. The *row* dictionary makes it easy to look up which population points are covered by a certain facility, but it is very inefficient to determine which facilities can cover a specific population point. Therefore we create a second dictionary, *col*, which saves the same information, but structured in a different way. Each key is a population point, and the array that belongs to it contains all the potential facilities IDs that can cover that point. Below the *row* and *col* dictionary are shown for the example that we introduced in the previous section.

$$B = \overbrace{\begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}}^{n} \quad row = \begin{Bmatrix} 0: & [0,3,6] \\ 1: & [1,2] \\ 2: & [1,3,5] \\ 3: & [3,5] \\ 4: & [4] \end{Bmatrix}, \quad col = \begin{Bmatrix} 0: & [0] \\ 1: & [1,2] \\ 2: & [1] \\ 3: & [0,2,3] \\ 4: & [4] \\ 5: & [2,3] \\ 6: & [0] \end{Bmatrix}$$

In our notation throughout this paper, $row[f]$ represents the array with population point IDs that are covered by facility f, and $col[i]$ represents the array with facility IDs that can cover population point i. The information in the dictionaries can only be accessed by iterating over the keys, so logical indexing does not work anymore. Therefore, we do not store the solution as the binary vector y, but we store the solution as an array with the active facility IDs, which we name F^*.

$$y = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \rightarrow F^* = \begin{pmatrix} 0 & 2 \end{pmatrix}$$

Before continuing, we would like to check whether the introduction of *row* and *col* indeed solves the memory issue, and whether they are small enough such that we can keep both dictionaries. Table 3 shows the total memory usage of the two dictionaries for each Vietnam case that we consider. We will run the heuristic on a 16 GB RAM computer, so from the table we conclude that we should not encounter any memory problems, as long as the rest of the heuristic is not too heavy on the memory. Therefore we choose to stick with using both *row* and *col*, as this allows for a more efficient implementation of the heuristic.

grid-size	$S = 20$	$S = 50$	$S = 100$
10	0.04	0.07	0.11
5	0.05	0.18	0.40
1	0.51	4.30	9.70

Table 3: Total memory usage of *row* and *col* in GB

Although the sparse matrix implementation with dictionaries was primarily meant to save memory, it also allows us to adjust the *cov* vector and calculate the objective change in a more efficient way. We explained in section 5.1 that with the old approach, it takes $O(n)$ time to calculate the objective change if a facility f_e were to enter the solution. We will explain now how we can use the sparse matrix implementation to improve on this complexity. In the new approach, we check for each point in $row[f_e]$ if its coverage value is equal to zero, and collect the points for which this is true in a set named $\mathcal{IN}$. These are the points that contribute to the objective increase caused by f_e entering the solution, because they were not yet covered in the old solution, but will be in the new solution. After constructing $\mathcal{IN}$, we increase the *cov* value by one for all of the points in $row[f_e]$.

The mathematical formulation of $\mathcal{IN}$ is as follows: $\mathcal{IN} \leftarrow \{i \in row[f_e] : cov(i) = 0\}$. Note that in the code, $\mathcal{IN}$ is actually a Numpy array and not a set, to increase compatibility with the other data structures. Even so, we have chosen for the set notation to improve readability, and because $\mathcal{IN}$ actually represents a set of points. The values in $\mathcal{IN}$ are the population points that are newly covered by the facility f_e, because if they were already covered in the old solution, the coverage value would be at least 1. This means that the objective increase can be calculated by summing $pop(i)$ for all $i \in \mathcal{IN}$. Following a similar reasoning, $\mathcal{OUT} \leftarrow \{i \in row[f_l] : cov(i) = 1\}$ contains all population points which become uncovered if f_l leaves the solution. Summarizing, the following two formulas can be used to calculate the objective change caused by a facility leaving or entering the solution:

$$\Delta obj = \sum_{i \in \mathcal{IN}} pop(i) \qquad \text{with } \mathcal{IN} \leftarrow \{i \in row[f_l] : cov(i) = 0\}$$

$$\Delta obj = -\sum_{i \in \mathcal{OUT}} pop(i) \qquad \text{with } \mathcal{OUT} \leftarrow \{i \in row[f_l] : cov(i) = 1\}$$

To illustrate this process, we will show the steps that follow facility 2 entering our example solution, which only contains facility 0 so far:

$$
\begin{aligned}
F^* &= \begin{pmatrix} 0 \end{pmatrix}, f_e = 2 \rightarrow F^* = \begin{pmatrix} 0 & 2 \end{pmatrix} \\
row[2] &= [1, 3, 5] \\
cov &= \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \\
\mathcal{IN} &= \{i \in [1, 3, 5] : cov(i) = 0\} \\
&= \{cov(1) = 1 \stackrel{?}{=} 0, cov(3) = 2 \stackrel{?}{=} 0, cov(5) = 0 \stackrel{?}{=} 1\} = \{1, 5\} \\
cov &= \begin{pmatrix} 1 & 0^{+1} & 0 & 1^{+1} & 0 & 0^{+1} & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 1 & 0 & 2 & 0 & 1 & 1 \end{pmatrix} \\
\Delta obj &= pop(1) + pop(5) = 10 + 30 = 40
\end{aligned}
$$

Let k be the number of elements in $row[f_e]$, then the complexity of updating the coverage vector is equal to $O(k)$, because only k additions are executed. Determining which elements are in $\mathcal{IN}$ also takes $O(k)$ time, because the coverage value is only checked for the k values in $row[f_e]$. Calculating the Δobj again consists of summing a maximum of k values, so the whole process has complexity $O(k)$. If we redefine k as the maximum length of $row[f]$ for any $f \in \mathcal{M}$, we can state that any addition, removal or swap of facilities has complexity $O(k)$, following the same reasoning as above. The fact that we have sparse data implies that $k \ll n$, so the complexity of $O(k)$ is a substantial improvement compared to $O(n)$, which was the complexity of calculating an objective change with the old approach.

5.3 Construction phase

We have established the basic data structures that we will be working with throughout our heuristic, and how they relate to the mathematical formulation of MCLP. This allows us to continue with the first phase of any GRASP heuristic, the construction phase. We will first look into the fully greedy approach, highlighting all the steps we took to make our implementation as efficient as possible. Once this method is understood, it is easy to make small adaptations which will introduce randomness into the greedy solution. We will explain the various ways in which we combine randomness and greediness, such that we obtain the construction methods discussed in section 3.2.

5.3.1 Greedy approach

Using a greedy approach can be very slow if not implemented in a smart way. As we have seen in the theoretical background, the worst-case complexity of choosing the facilities in a greedy way is $O(pmn)$. Calculating the greedy metric for each facility takes $O(n)$, which is done for each of the m potential facilities, in each of the p iterations, so this results in the complexity $O(pmn)$. However, if the greedy algorithm is implemented in a smarter way, there is no need to perform all of these computations in every step. We have seen in the previous section that we can exploit the sparsity of the data to calculate the objective change caused by a facility entering the solution, which is the same as calculating the greedy value of a facility, in $O(k)$ time. Furthermore, for many potential facilities the greedy metric does not change from one iteration to the next, so we only need to recalculate this metric if it changes. If we make use of these advantages, the running time reduces drastically. In this section we explain how to implement these improvements for the greedy algorithm.

To measure the impact of these improvements, we first report the run-time of a naive implementation of the greedy algorithm. We report the run-times on the case with $S = 50$, $grid\text{-}size = 10$, and $p = 50$, which implies $m = 3555$ and $n = 302,046$. The naive implementation does not use any of the tricks to accelerate the algorithm. It uses the binary matrix B, and calculates the greedy metric for every potential facility from scratch at every iteration. For the test case, it takes 289 seconds to calculate the greedy solution, averaged over 10 runs.

As a first step, we wanted to check whether using the *row* and *col* dictionaries as sparse matrices was a wise choice. If their structure would complicate the implementation too much, we would try an alternative representation of the sparse matrices. Replacing the binary matrix B by the dictionaries was fairly straightforward however, and even resulted in an acceleration of finding the greedy solution in an average of 133 seconds. Note that this was still without utilizing the sparsity of the data as discussed in section 5.2, so calculating the greedy metric for a facility still took $O(n)$ time.

The next major improvement in running time was achieved by exploiting the fact the greedy value of a facility is often not affected when another facility is added to the solution. For example, if a facility in the south of Vietnam is chosen, it will not affect the greedy values of the potential facilities in the North. In fact, the greedy value of a facility f only changes if there is overlap between the population points that are covered by f, and the population points that are uniquely covered by the chosen facility f_{sel}. If this is the case, the greedy value of facility f will decrease, because there is no added value in covering a population point by more than one facility.

There are two ways to determine the new greedy values of the affected facilities. The first option is to iterate over all population points that are newly covered by f_{sel}, and adjust the greedy values of the other facilities that could have covered this point by the appropriate amount. The second method is to find the set of affected facilities, and simply recalculate the greedy value from scratch for each of these affected facilities. This second option turned out to be faster, so we opted for this method.

With the help of both the *row* and *col* dictionary, we find the list of affected facilities. Once the coverage vector has been updated, the set $\mathcal{IN}_{sel} \leftarrow \{i \in row[f_{sel}] : cov(i) = 1]\}$ contains all population points that are uniquely covered by the last selected facility. For each point in $\mathcal{IN}_{sel}$, we look up which facilities can cover it in the *col* dictionary, and concatenate the corresponding array to Z. Z is the array with all the facility IDs for which the greedy metric changes. Many facility IDs will occur multiple times in Z, thus after iterating over all points in $\mathcal{IN}_{sel}$, we remove all duplicates, ending up with the desired list of affected facilities. As a result of only recalculating the greedy value for the facilities for which it is necessary, we reduced the running time from 133 seconds to 10 seconds. Despite this big reduction of the run-time, we can see in line 8 of Algorithm 1 that the worst-case efficiency of the greedy algorithm is not affected by this trick, because in theory all m facilities can occur in Z. In practice this is never the case however, which explains the observed time reduction.

Next, we conducted some small technical improvements in the code that enabled us to find the greedy solution in 5 seconds, again averaged over 10 runs. We do not want to delve too much into the technicalities of the code, but the biggest gain was achieved by rewrit-

ing all code such that it only uses Numpy functions and Numpy arrays to store the vectors.

The last big improvement was utilizing the sparsity of the data, following the concept that we discussed in section 5.2. With this improvement, computing the greedy metric is reduced from $O(n)$ to $O(k)$ time, with $k = \max_{f_e} |row[f_e]|$, because the coverage values are only checked for the population points in $row[f_e]$. Another advantage of this trick is that we do not need to update the coverage vector before calculating the greedy value of a candidate facility. It is valuable that *cov* can be used without updating it, because many greedy values have to be calculated before picking the best one. Now, the same *cov* vector can be used for all of these calculations, instead of increasing and decreasing the coverage values every time. *cov* will only be updated once the best facility is selected and actually added to the solution. By using the sparsity of the data to our advantage, we managed to further decrease the running time from 5 to 0.5 seconds.

In algorithm 1, the pseudo-code of the resulting greedy construction algorithm is given. In the comments, we have indicated the time complexity of every line, however we have omitted $O(1)$ for improved readability. The indentation of $O(*)$ reflects the indentation of the loop in which the line is executed, so you can see how often each line has to be executed. We will use these comments to analyze the complexity of the whole algorithm. Finally, $k = \max_f |row[f]|$ is the maximum number of population points that a potential facility can cover, and $l = \max_i |col[i]|$ is the maximum number of facilities by which a population point can be covered. Note that the sparsity of the data implies $k \ll n$ and $l \ll m$.

Algorithm 1: Greedy Function

```
Function Greedy(M, N, pop, row, col, p)
    Z: Array with all potential facilities for which the greedy metric changes
    IN: Set of additionally covered population points

    Initialize greedy and cov as zero vectors of length |M| and |N| respectively
    Initialize obj ← 0, Z ← M, and F* as an empty list
    for iter = 1, ..., p do                                   // O(p)
        for f ∈ Z do                                          //   O(m)
            IN ← {i ∈ row[f] : cov(i) = 0}                    //     O(k)
            greedy(f) ← Σ_{i∈IN} pop(i)                       //     O(k)
        end
        f_sel ← argmax_f greedy(f)                            //   O(m)
        Append selected facility f_sel to solution F*
        Update obj: obj ← obj + greedy(f_sel)
        Update cov: let cov(i) ← cov(i) + 1  ∀i ∈ row[f_sel]  //   O(k)
        Reset Z to an empty array
        IN_sel ← {i ∈ row[f_sel] : cov(i) = 1}                //   O(k)
        for i ∈ IN_sel do                                     //   O(k)
            Concatenate all elements in col[i] to Z           //     O(l)
        end
        Remove duplicate values from Z                        //   O(kl)
    end
    return F*, cov, obj
end
```

Looking at the worst case complexity, we can partly explain the improved performance of our implementation compared to the naive implementation. Summing all complexities, we obtain $O(p(m2k + m + 2k + 2kl))$. Constants will not dominate the time complexity as the problem grows, and can therefore be omitted. Furthermore, $l \ll m$ so it is allowed to replace l by m, resulting in $O(pmk)$. Given the fact $k \ll n$, this is a substantial improvement compared to the usual worst case complexity of constructing the maximum covering greedy solution, which is $O(pmn)$.

5.3.2 Combine randomness and greediness

Now that we have outlined how a fully greedy solution can be constructed efficiently, we will show how this base algorithm can be adjusted in order to get the various construction techniques that we discussed in section 3.2. Each of these techniques combine randomness and greediness in a certain way, aiming to generate high quality solutions with sufficient variation between them.

- *randomized greedy (rgreedy)*: The only thing that needs to be adjusted from the fully greedy approach is the selection criterion in line 12 of Algorithm 1. Once the *greedy* vector has been computed, sort the values in descending order. Make sure to save which index the sorted values had in the original *greedy* vector. Then uniform randomly select one of the top $\lfloor \alpha * p \rfloor$ indices, this will be the selected facility f_{sel}. This facility will be added to the solution, and the rest of the algorithm continues unchanged.

- *random plus greedy (rpg)*: After line 5 and 6 of Algorithm 1, we introduce some intermediate steps. The first $\lfloor \alpha * p \rfloor$ facilities are chosen at random from the list of potential facilities. These facilities are added to F^*, then *cov* and *obj* are updated accordingly. Nothing is altered in the rest of the algorithm, except that line 7 will iterate from 1 up to $p - \lfloor \alpha * p \rfloor$, such that in total p facilities are selected.

- *proportional greedy (pgreedy)*: We make the same adjustments as in *rgreedy*, except that the random selection has a different underlying distribution, which biases the selection towards facilities with a higher rank. There are several bias functions, as described in section 3.2, and they can be used to assign a probability to each of the top $\lfloor \alpha * p \rfloor$ facilities. The selected facility f_{sel} will be chosen at random from these facilities, according to the customized probability distribution.

- *sample greedy (sample)*: This approach differs the most from the standard greedy algorithm. The greedy vector needs to be reset as a zero vector at the beginning of each iteration. Then, Z is filled with a random sample of q elements from $\mathcal{M}$. For these q elements, the greedy value is calculated, the best facility is selected, and F^*, *obj* and *cov* are updated. The steps in line 16 up to and including 21 in Algorithm 1 are not executed. At each iteration we just calculate the q greedy values of the facilities in the random sample, so determining which greedy values of the other facilities would be affected, is irrelevant.

These are all the construction methods that we will consider. We will not test the *random* construction method, because empirically it has often been shown that the quality of fully random solutions is too low.

5.4 First improvement local search

Once a solution has been constructed, the local search phase starts. In this section we will discuss a first improvement local search, in the next section we will elaborate on the best improvement strategy. While developing the model, we found that the local search can be a valuable, but also very costly tool. Let us consider the same case as in the previous section, using a 10 km grid for the facility locations, $p = 50$ new facilities, and $S = 50$ as the covering distance. The naive implementation of local search, using the binary matrix B and calculating objective changes as in section 5.1, took 85 minutes to arrive at a 1-swap local optimum. Clearly, it would not be feasible to apply this approach to the even larger cases. Fortunately, many of the improvements that we implemented for the greedy algorithm can be applied in the local search as well.

In Algorithm 2 we show the basic structure of a first improvement local search. For each facility f_l in the current solution F^*, the objective decrease w_{loss} associated with f_l leaving the solution is calculated. This is compared to the objective increase w_{gain} caused by f_e entering the solution, taking the place of f_l. For a given leaving solution, each facility in the set of candidates is considered. The set of candidates C consists of all potential facilities that are not in the current solution. As soon as we find a profitable swap, i.e. $w_{gain} > w_{loss}$, we execute the swap and restart the search. We also show the time complexity of each line, so we can determine the time complexity of 1 iteration. Given that an objective change can be calculated in $O(k)$ time, as explained in 5.2, the total time complexity is $O(pmk)$.

Algorithm 2: Basic First Improvement Local Search

```
1  Function Local_Search(F*, C)
2      for f_l ∈ F* do                                          // O(p)
3          Compute objective decrease w_loss by f_l leaving F*  //  O(k)
4          for f_e ∈ C do                                       //  O(m)
5              Compute objective increase w_gain by f_e entering F*\{f_l}  //  O(k)
6              if w_gain > w_loss then
7                  F* = F* ∪ {f_e}\{f_l}
8                  C = C ∪ {f_l}\{f_e}
9                  Return to line 2
10             end
11         end
12     end
13 end
```

w_{gain} and w_{loss} can be calculated efficiently using the technique discussed in 5.2. The sum of the population of all points in $\mathcal{OUT} \leftarrow \{i \in row[f_l] : cov(i) = 1\}$ is equal to w_{loss}, and similarly, all points in $\mathcal{IN} \leftarrow \{i \in row[f_e] : cov(i) = 0\}$ contribute to w_{gain}. An important step in the local search process, is that the coverage vector needs to be updated after calculating w_{loss}, as if f_l already left the solution. With the new coverage vector as a starting point, calculating w_{gain} of a candidate facility is equivalent to calculating its greedy value. However, if w_{gain} is smaller than w_{loss} for all $f_e \in C$, f_l will stay in the solution, so the change of the coverage vector needs to be undone, and the local search continues with the next facility in F^*.

If F^* and C are searched in the same order each time, the same swaps are considered many times. Strictly speaking, whether a certain swap is profitable also depends on the solution F^*, so the same swap might not be profitable the first time you consider it, but it could become profitable as the incumbent solution changes. However, in each step of the local search only 1 facility changes in the solution, so it might be more efficient to focus on examining new swaps. A simple way to make sure that the solution and candidate list are not searched in the same order each time, is to apply a random shuffle to F^* and C each time a new solution is found. Using the sparse matrix implementation, exploiting the sparsity to efficiently calculate objective changes, and applying a random shuffle to F^* and C^* after each swap, accelerates the first improvement local search substantially. The adapted algorithm finds a local optimum in 40 seconds instead of in 85 minutes.

Even though this is already a huge improvement, we would like to speed up the process even more, such that the local search can also be executed on the larger cases. We decided to reduce the neighbourhood of each facility, by only allowing swaps with facilities which are relatively close. As a result, less swaps have to be considered, which accelerates the local search. On the other hand, the fact that we consider fewer swaps could also result in a lower local optimum. We take the following steps to determine the restricted neighbourhood: let f_l denote a facility that is considered to leave the solution, then calculate the haversine distance from f_l to all other facilities, and only keep the ones which are located within *border* km. Next, sort the remaining candidates according to their distance to f_l, such that the local search will start with the closest facility.

In section 6.1.2, we will analyse the effect of the *border* value on the speed and the quality of the local optimum in several problem settings. From some preliminary experiments, we already observe that the restriction of the neighbourhood can be very valuable. When applied to the greedy solution of our reference case, with a border of 50 km, the local search finds the same local optimum as without the restriction, in just 2.2 seconds.

There are two more additions that we made to the algorithm. The first one, is that we store the restricted candidate list of each facility in a dictionary C_{dict}. At the start of the algorithm, we calculate and save the candidate list for each facility in solution F^*. Then each time a swap is executed, we add the candidate list of the entered facility to the dictionary. Using the dictionary saves computation time, because we only determine which candidates are close enough once. The second extension, is the vector *no_improv*, which keeps score of how many times the restricted candidate list of a given facility has been searched without success. After the random shuffle of F^*, we sort the facilities in the solution according to their corresponding values in the *no_improv* vector, such that the facility which has been researched the least is prioritized. The introduction of C_{dict} and the *no_improv* vector result in an even shorter solving time of 0.5 seconds.

The whole algorithm, including the extensions we introduced, is written in pseudo-code in Algorithm 3. Just as in the previous algorithms, we have given the time complexity of each line, so we are able to examine whether the improved speed of the algorithm is also reflected in the worst-case complexity. We define z as the maximum number of facilities that lie within the *border* of any of the facilities. The size of z compared to m depends on the value chosen for *border*, but we assume that we choose *border* in such a way that $z \ll m$. Consider the 5km grid data with $border = 50$ for example, then $z = 349$, which is 40 times smaller than $m = 14,235$.

Algorithm 3: Local Search Function

```
1  Function Local_Search(F*, cov, obj, M, pop, row, border)
2      d_le: haversine distance between facility f_l and facility f_e
3      OUT: Set of additionally uncovered population points
4      C_dict: Dictionary, stores border restricted candidate list for a given facility
5      no_improv: Counts # unsuccessful searches of each facility's candidate list
6
7      flag = True
8      Initialize no_improv as a zero vector of length |M|
9      for f_l ∈ F* do                                                    // O(p)
10         Calculate d_le ∀f_e ∈ M                                        // O(m)
11         C_dict[f_l] ← {f_e ∈ M : d_le ≤ border}                        // O(m)
12     end
13     while flag = True do
14         flag ← False
15         First random shuffle F*, then sort according to no_improv      // O(p)
16         for f_l ∈ F* do                                                // O(p)
17             C ← C_dict[f_l]                                            // O(z)
18             Sort C in increasing order according to distance d_le      // O(z)
19             OUT ← {i ∈ row[f_l] : cov(i) = 1}                          // O(k)
20             w_loss ← Σ_{i∈OUT} pop(i)                                  // O(k)
21             Update cov: cov(i) ← cov(i) − 1 ∀i ∈ row[f_l]              // O(k)
22             for f_e ∈ C do                                             // O(z)
23                 IN ← {i ∈ row[f_e] : cov(i) = 0}                       // O(k)
24                 w_gain ← Σ_{i∈IN} pop(i)                               // O(k)
25                 if w_gain > w_loss then
26                     flag ← True
27                     obj ← obj + w_gain − w_loss
28                     Update cov: cov(i) ← cov(i) + 1 ∀i ∈ row[f_e]      // O(k)
29                     Delete f_l from F* and insert f_e
30                     Calculate d_ek ∀k ∈ M                              // O(m)
31                     C_dict[f_e] ← {f_k ∈ M : d_ek ≤ border}            // O(m)
32                     Return to line 9
33                 end
34             end
35             if flag = False then
36                 no_improv(f_l) ← no_improv(f_l) + 1
37                 Undo change in cov: cov(i) ← cov(i) + 1 ∀i ∈ row[f_l]
38             end
39         end
40     end
41     return F*, cov, obj
42 end
```

The local search algorithm has an initialization cost of $O(pm)$, which is needed to construct the candidate dictionary for the facilities in the starting solution. We would also like to note that the complexity indentation of lines 28, 30 and 31 shows that you pass these lines only once per local search iteration. To see why this is true, recall that a swap is

executed as soon as the algorithm finds an improving swap, which finishes that local search iteration. From the complexities in Algorithm 3 we conclude that 1 local search iteration has worst-case complexity $O(pzk + m)$, with a one-time initialization cost $O(pm)$. Given that $z \ll m$, this is a substantial improvement if we compare it to the basic local search algorithm without the restriction on distance, which had complexity $O(pmk)$.

5.5 Best improvement local search

In this section we will discuss two ways to implement a best improvement local search. The first one is the simple approach, which only requires a few adaptations to our first improvement local search in Algorithm 3. The alternative method is not as straightforward and requires a more extensive explanation, therefore we will refer to it as the advanced approach.

The alterations needed to transform Algorithm 3 into a best improvement local search are trivial. The random shuffle of F^* and the *no_improv* vector are removed, because the order in which you consider the swaps is not relevant anymore. Furthermore, the candidate list C consists of all potential facilities not in the current solution, without a restriction on the distance. Finally, instead of executing a profitable swap immediately, the algorithm keeps track of the best swap, which is executed once all possible swaps have been considered. The rest of the set-up of this algorithm remains the same.

The strength of our own implementation of the first improvement local search is that we try to minimize executing unpromising calculations. To this goal, we restrict the neighbourhood of each chosen facility, and prioritize facilities which have been investigated the least. However, these advantages are lost once our approach is transformed into a best improvement local search. Considering all swaps in every iteration implies that many calculations are repeated in subsequent iterations, so it takes longer to reach a local optimum. Ideally, we would like to eliminate these repetitive computations, which is where the advanced approach comes in.

In the advanced approach, we first calculate the objective change corresponding to all possible swaps, and save these outcomes. Then we pick the best one, and determine which of other outcomes are affected by the chosen swap. After updating these affected values, we repeat this process, until a local optimum is found. Note the similarity to our greedy algorithm, in which we calculate the greedy metric for every facility only once, and update the affected greedy values as facilities are added to the solution. To implement this idea for a local search is more complicated than for the greedy algorithm however, as it considers swaps instead of only adding facilities.

As mentioned in section 3.3, Resende and Werneck [23] were the first ones to implement a best improvement local search in this advanced way. Their algorithm is written for a p-median problem, so we have to translate it to be applicable to the maximal covering problem. To the best of our knowledge, there do not exist any papers which discuss how this translation from PMP to MCLP can be accomplished. As this translation is far from trivial, the rest of this section will focus on how to implement the best improvement local search for a MCLP, using the advanced approach. Algorithm 4 shows the resulting algorithm, which can be used as a reference throughout this section. All parts of the pseudo-code should be clear after our step by step explanation of the process.

5.5.1 Constructing *loss*, *gain*, and *extra*

This implementation makes use of three auxiliary data structures, in which certain calculations are saved, such that these do not have to be repeated. The first structure is the *loss* vector, each item in the *loss* vector reflects the added value of that facility in the current solution. In other words, it is the amount with which the objective would decrease if that facility would be removed from the current solution, and we already know from section 5.2 how we can calculate this value efficiently using the set $\mathcal{OUT}$. The second structure is the *gain* vector, which indicates for each potential facility which is not in the current solution by how much the objective would increase if that facility would be added. This vector is identical to the greedy vector, and each entry is computed using $\mathcal{IN}$. For the sake of completeness of the explanation of the best improvement local search implementation, the loss and gain values are computed as follows:

$$loss(f_l) = \sum_{\{i \in row[f_l]: cov(i)=1\}} pop(i)$$

$$gain(f_e) = \sum_{\{i \in row[f_e]: cov(i)=0\}} pop(i)$$

Below the *loss* and *gain* vector are given for the example that we have used throughout this section. The numbers on the right of the dashed line indicate to which facilities the values correspond. Take facility 0, this facility covers population points 0, 3 and 6. Of these population points, 3 is also covered by facility 2 in the current solution, so only the number of inhabitants belonging to 0 and 6, which is 20 and 8, contribute to the value in *loss*. The *loss* value for facility 2 can be computed similarly. Next, we consider facility 1 in the *gain* vector, it could cover population points 1 and 2 if it would be added to the solution. However, point 1 is already covered by facility 2, so the *gain* value only reflects the number of inhabitants of population point 2, which is 8. The rest of the *gain* vector can be filled likewise.

$$loss = \begin{array}{c} \\ \\ \end{array}\left\{\begin{pmatrix} 20+8 \\ 10+5 \end{pmatrix}\right. \begin{array}{l} 0 \\ 2 \end{array} \qquad gain = \begin{array}{c} \\ m-p \end{array}\left\{\begin{pmatrix} 8 \\ 0 \\ 12 \end{pmatrix}\right. \begin{array}{l} 1 \\ 3 \\ 4 \end{array}$$

Using the *loss* and *gain* vector, it is easy to calculate the effect of adding or removing a facility to or from the current solution, but this is not the same as executing these two actions simultaneously, i.e. executing a swap. To understand this concept, consider swapping facilities 2 and 1 in our example. Facility 2 covers population point 1 uniquely, so the corresponding 10 people contribute to its *loss* value. As this point is already covered in the current solution, it does not contribute to the *gain* value of facility 1, even though this facility could also cover population point 1. Indeed, if facility 0 would be swapped for facility 1, it would have no extra value that facility 1 covers this point. However, because it is facility 2 that is leaving the solution, population point 1 would have been uncovered, if was not for facility 1 covering it in the new solution. In this case it is valuable that facility 1 covers population point 1, but this is not visible from the *gain* value. There clearly exists interaction between leaving and entering facilities which has effect on the objective change caused by a swap. This interaction term is captured in the third auxiliary structure, *extra*.

The formula used to compute the *extra* value for a given entering facility f_e and leaving facility f_l, is shown below. The sum is taken over all population points in $\mathcal{OUT}$, so they are uniquely covered by the leaving facility in the current solution. These points will become uncovered unless the entering solution covers them, and that is exactly what

the *extra* value should capture. For each of these population points i we check if the entering facility f_e can cover this population point, which is the case if f_e occurs in $col[i]$. Summarizing, the *extra* value is the sum of the number of inhabitants that correspond to the population points that are uniquely covered by f_l, and can be covered by f_e. Note that this interaction term can never be negative.

$$extra(f_e, f_l) = \sum_{\{i \in row[f_l]: cov(i)=1\}} \mathbb{1}_{\{f_e \in col[i]\}} * pop(i)$$

All the *extra* values are collected in a matrix of size p by $m - p$. The matrix displays the interaction term for each pair of facilities that can be swapped. Below the *extra* matrix for our simple example is depicted, with under and to the right of the dashed lines the facilities to which the numbers correspond. Facility 0 has no interaction with the candidate facilities that can be swapped in. Population points 0 and 6 can only be covered by facility 0, so for sure they do not cause any interaction. Population point 3 is covered by facility 0, but also by facility 2 in the current solution. Regardless of the facility that leaves the solution, population point 3 will stay covered, so it if the entering facility also covers this point, this has no added value. We already discussed the interaction between facility 2 and 1, which is also visible in the matrix now. Similarly, replacing facility 2 by 3 results in an interaction term caused by population point 5.

$$extra = \begin{array}{c} \\ \\ \end{array}\begin{array}{c} \overbrace{\begin{pmatrix} 0 & 0 & 0 \\ 10 & 5 & 0 \end{pmatrix}}^{m-p} \begin{array}{c} 0 \\ 2 \end{array} \\ \begin{array}{ccc} 1 & 3 & 4 \end{array} \end{array}$$

Once *loss*, *gain* and *extra* have been computed, the profit formula given below can be used to calculate which swap generates the best profit. It is equal to the added value of the entering facility, minus the lost value of the leaving facility, plus the interaction term of these two facilities. Let f_l^* and f_e^* be the leaving and entering facility-pair that maximizes this profit function for all possible swaps. If $profit(f_l^*, f_e^*)$ yields a profit larger than 0, the swap will be executed. Otherwise, there exists no swap which improves the objective function, which means that the current solution is a local optimum, and the best improvement local search is finished.

$$profit(f_l, f_e) = gain(f_e) - loss(f_l) + extra(f_l, f_e)$$

Turning back to our example, the profit matrix can be generated from the previously computed data structures, and is given below. We observe there is only one swap that results in a positive profit, thus facility 2 will leave the solution such that facility 1 can enter.

$$profit = \begin{array}{c} \begin{pmatrix} -20 & -28 & -16 \\ 3 & -10 & -3 \end{pmatrix} \begin{array}{c} 0 \\ 2 \end{array} \\ \begin{array}{ccc} 1 & 3 & 4 \end{array} \end{array}$$

5.5.2 Solving discrepancy between indices and facility IDs

In our explanation until now, we have assumed that it is no problem to translate a facility number into the correct index of a data structure. For example, if we want to know the

extra value for $f_l = 2$ and $f_e = 1$ in our own illustrative case, we can simply look up $extra(2, 1) = 10$. However, in coding language in this is not so straightforward. We use Numpy arrays to store all the large data structures, and the fastest way to access and alter values is via their indices. Thus, in our code we would need to look up $extra(1, 0)$ to get the desired *extra* value for $f_l = 2$ and $f_e = 1$. Clearly, there exists a discrepancy between the indices in the data structures and the facility ID they represent.

In order to deal with this discrepancy, we make some adjustments. First, the width of the *extra* matrix and the length of the *gain* vector is increased to m, instead of $m - p$. The altered versions of these data structures are shown below. In our application $p << m$, so the increase of memory usage is negligible. The big advantage is that and there exists no more discrepancy in *gain* and in the columns of *extra*. As we will explain later on, the facility IDs in *col* will be used to indicate which columns in *extra* and which rows in *gain* are affected by a swap. Being able to skip the step of translating the values in *col* to their corresponding indices improves the speed of the algorithm significantly.

$$gain = \begin{array}{c} \\ \\ \\ \\ \end{array}\left\{\begin{array}{c} \begin{pmatrix} 0 \\ 8 \\ 0 \\ 0 \\ 12 \end{pmatrix} \end{array}\right. \begin{array}{c} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \qquad extra = \left\{\begin{array}{c} \overbrace{\begin{pmatrix} 28 & 0 & 0 & 0 & 0 \\ 0 & 10 & 15 & 5 & 0 \end{pmatrix}}^{m} \\ \begin{array}{ccccc} 0 & 1 & 2 & 3 & 4 \end{array} \end{array}\right. \begin{array}{c} 0 \\ 2 \end{array}$$

We could solve the discrepancy for in the *loss* vector and in the rows of the *extra* matrix in the same way, but this would not be feasible in terms of memory usage. The *extra* matrix would need $m \times m$ data points, and we would run into the same memory issues as discussed in section 5.2. Therefore, we have come up with an alternative way to solve this issue. Constructing *loss* and *extra* for the first time, can simply be done row by row. Updating them after a swap is more complicated, and will be explained in detail later on. For now we just note that our method needs the vector ϕ. This vector indicates for every uniquely covered population point, the index of its covering facility in the current solution. The ϕ corresponding to our example is illustrated below.

$$\phi = \begin{pmatrix} 0 & 1 & * & * & * & 1 & 0 \end{pmatrix}$$

5.5.3 Updating *loss*, *gain* and *extra*

Once a swap has been executed, the auxiliary data structures have to be updated. Recomputing everything from scratch is an inefficient option, because many values do not change from one iteration to the next. Only changing the affected values and reusing the rest of the computations saves a lot of time, and this is exactly what makes this method faster than our own implementation of best improvement local search. There are four cases which cause a value to change in one of the data structures, which we will discuss one by one:

1. A population point was first covered by two facilities, in the new solution by one.
2. A population point was first covered by one facility, in the new solution by two.
3. A population point was first covered by one facility, in the new solution by none.
4. A population point is uniquely covered by the entered facility

We collect all the points that belong to the first case in a vector named *cov_21*. The decrease in coverage can only be caused by the leaving facility. Therefore, to find the population points that belong to *cov_21*, we first restrict to the points that were covered by the leaving facility: $row[f_l^*]$. Among these, we select the ones that are covered by only one facility in the new solution, thus: $\{i \in row[f_l^*] : cov(i) = 1\}$. Finally, of these points we discard the ones that are covered by both the leaving and entering facility. We collect the population points that are covered by both facilities in *overlap*: $overlap = \{row[f_e^*]\} \cap \{row[f_l^*]\}$. If a point is in *overlap* and has single coverage in the new solution, it was uniquely covered by the leaving facility in the old solution and is uniquely covered by the entering facility in the new solution. Thus its coverage did not decrease from two to one, so it should not be in *cov_21*. Summarizing:

$$cov_21 = \{i \in row[f_l^*] : [cov(i) = 1] \wedge [i \notin overlap]\}$$

Consider a population point $i \in cov_21$ for which the coverage decreases from 2 to 1 by the executed swap. The *loss* value of the facility that still covers i will increase with $pop(i)$, because it is the only remaining facility covering point i. As i is now uniquely covered, $\phi(i)$ gives the index of this covering facility in F^*, which allows us to define $f_a = F^*(\phi(i))$ as the affected facility. $\phi(i)$ is also the index of the value that needs to be adjusted in *loss*, as this is the loss value of f_a. Furthermore, $phi(i)$ also gives the index of the affected row in the *extra* matrix, because the interaction term of f_a with all the facilities in $col[i]$ increases by $pop(i)$. To understand this, recall that all facilities which can cover point i are found in $col[i]$. If any of these facilities would be swapped in the next iteration with f_a, the entering facility would take overtake the unique covering position of i from f_a, thus $pop(i)$ is exactly the interaction term.

We name the collection of points for which the coverage increases from one to two, *cov_12*. In this case, the points have to be covered by the entering facility, and their updated coverage value has to be equal to two. Following a similar reasoning as in the previous case, the points in *overlap* are excluded because these were already covered by two facilities in the old solution, not one. We thus obtain the following equation:

$$cov_12 = \{i \in row[f_e^*] : [cov(i) = 2] \wedge [i \notin overlap]\}$$

Again, the *loss* vector and *extra* matrix are affected. In order to know which row we have to adjust, we need ϕ of the old solution. Therefore, we save $\phi_{old} \leftarrow \phi$ at every step, before we update ϕ to coincide with the new solution. For a point i in *cov_12*, $\phi_{old}(i)$ indicates the index of the facility that was the unique covering facility of i in the old solution, and we define $f_a = F^*(\phi_{old})$. In the old solution $pop(i)$ contributed to the loss value of f_a and to the interaction term of f_a with all other facilities that can cover i. In the new solution this is no longer the case, because f_a as well as the entering facility f_e^* can cover i. To process this, we subtract $pop(i)$ from $loss(\phi_{old}(i))$ and from all elements in $extra(\phi_{old}, col[i])$ for every i in *col_12*. There is no need to alter the *loss* value or *extra* row corresponding of f_e*, because these will be filled correctly as the final step of the updating process.

In the third group of population points are points that were uniquely covered in the old solution and are not covered at all in the new solution. Clearly, these points have to be in $row[f_l^*]$ and will not occur in *overlap*. Therefore, all points in $row[f_l^*]$ for which the new coverage is zero belong to this group:

$$cov_10 = \{i \in row[f_l^*] : [cov(i) = 0]\}$$

The *loss* value and *extra* row belonging to f_l^* will be overwritten by the loss value of f_e^*, so we leave them untouched for now. The *gain* vector is affected however, because in the old solution it was worthless if a candidate facility could cover a population point $i \in cov_10$, because it was already covered. In the new solution this is no longer the case, so the *gain* value of all facilities that can cover point $i \in cov_10$, listed in $col[i]$, increases by $pop(i)$.

Lastly, we have the points that are uniquely covered by the entered facility in the new solution. The definition of this group differs from the other ones, because this group is used to fill the rows of *extra* and *gain* corresponding to facility f_e^* from scratch. The definition of cov_x1 in line 53 is equal to the definition of $\mathcal{OUT}$ in line 7, which implies we can fill the *extra* matrix in the same way as well. Note that we do not use cov_x1 to fill the *loss* value of f_e^*, because it is simply equal to its former gain value plus the interaction term which belonged to the executed swap. We saved this value in *aux* as a first step, so we do not need to recompute it. Finally, the *gain* vector can also be influenced by population points in cov_x1, but only if their coverage in the old solution was zero. To extract these points, we check whether a population point is in *overlap*. If not, the coverage must have increased from 0 to 1, thus the gain value of other facilities that can cover that point decreases.

This finalizes all the steps of the advanced approach to a best improvement local search, based on the implementation of Resende and Werneck. When applying this search to the greedy solution of our reference case, it finishes in 1.7 seconds, averaged over 10 runs. If we use the simple approach to execute the same search, it takes 58 seconds. The advanced approach results in such a big speed improvement, that it is feasible to consider the best improvement strategy as one of the options for the local search phase in the GRASP iterations. In the results chapter, we will compare the performance of the different options for the local search phase.

Algorithm 4: Best Improvement Local Search Function

```
1  Function Best_improvement_LS(F*, cov, obj, M, N, pop, row, col, p)
2      Initialize the vector of candidate facilities C ← M\{F*}
3      Initialize loss, gain and φ as zero vectors of length p, |M|, |N| respectively
4      Initialize extra as zero matrix of size p × |M|
5      Initialize F*_idx as an index vector of length p with values 0, ..., p − 1
6      for f_idx ∈ F*_idx and f ∈ F* do                          // O(p)
7          OUT ← {i ∈ row[f] : cov(i) = 1}                       //   O(k)
8          loss(f_idx) ← Σ_{i∈OUT} pop(i)                        //   O(k)
9          φ(i) ← f_idx ∀i ∈ OUT                                 //   O(k)
10         for i ∈ OUT do                                        // O(k)
11             extra(f_idx, col[i]) ← extra(f_idx, col[i]) + pop(i)   //   O(l)
12         end
13     end
14     for f ∈ C do                                              // O(m)
15         IN ← {i ∈ row[f] : cov(i) = 0}                        //   O(k)
16         gain(f) ← Σ_{i∈IN} pop(i)                             //   O(k)
17     end
18     improv ← True
```

```
While improv = True Execute vectorized profit function to obtain profit matrix: // O(pm)
    profit(f_l, f_e) ← gain(f_e) − loss(f_l) + extra(f_l, f_e) ∀f_l ∈ F*_idx, ∀f_e ∈ C
Let f*_{l,idx}, f*_{e,idx} be the coordinates of the highest value in profit        // O(pm)
Let f*_l ← F*(f*_{l,idx}) and f*_e ← C(f*_{e,idx}) be the corresponding facilities
if profit(f*_{l,idx}, f*_{e,idx}) > 0 then
    aux ← gain(f*_e) + extra(f*_{l,idx}, f*_e)
    Replace leaving by entering facility in current solution: F*(f*_{l,idx}) ← f*_e
    Replace entering by leaving facility in candidate vector: C(f*_{e,idx}) ← f*_l
    Update obj: obj ← obj + profit(f*_{l,idx}, f*_{e,idx})
    Increase coverage cov: cov(i) ← cov(i) + 1  ∀i ∈ row[f*_e]          // O(k)
    Decrease coverage cov: cov(i) ← cov(i) − 1  ∀i ∈ row[f*_l]          // O(k)
    φ_old ← φ                                                           // O(n)
    Reset φ to a vector of zeroes                                       // O(n)
    for f_idx ∈ F*_idx and f ∈ F* do                                    // O(p)
        OUT ← {i ∈ row[f] : cov(i) = 1}                                 //    O(k)
        φ(i) ← f_idx ∀i ∈ OUT                                           //    O(k)
    end
    overlap ← {row[f*_e]} ∩ {row[f*_l]}                                 // O(k)
    cov_21 ← {i ∈ row[f*_l] : [cov(i) = 1] ∧ [i ∉ overlap]}             // O(k)
    for i ∈ cov_21 do                                                   // O(k)
        loss(φ(i)) ← loss(φ(i)) + pop(i)
        extra(φ(i), col[i]) ← extra(φ(i), col[i]) + pop(i)              //    O(l)
    end
    cov_12 = {i ∈ row[f*_e] : [cov(i) = 2] ∧ [i ∉ overlap]}             // O(k)
    for i ∈ cov_12 do                                                   // O(k)
        loss(φ_old(i)) ← loss(φ_old(i)) − pop(i)
        extra(φ_old(i), col[i]) ← extra(φ_old(i), col[i]) − pop(i)      //    O(l)
    end
    cov_10 ← {i ∈ row[f*_l] : [cov(i) = 0]}                             // O(k)
    for i ∈ cov_10 do                                                   // O(k)
        gain(col[i]) ← gain(col[i]) + pop(i)                            //    O(l)
    end
    Reset extra row of leaving facility extra(f*_{l,idx}, :) ← 0        // O(m)
    cov_x1 ← {i ∈ row[f*_e] : [cov(i) = 1]}                             // O(k)
    for i ∈ cov_x1 do                                                   // O(k)
        extra(f*_{l,idx}, col[i]) ← extra(f*_{l,idx}, col[i]) + pop(i)  //    O(l)
        if i ∉ overlap then
            gain(col[i]) ← gain(col[i]) − pop(i)                        //    O(l)
        end
    end
    loss(f*_{l,idx}) ← aux
else
    improv ← False
end
return F*, cov, obj
end
```

In Algorithm 4, the time complexity of each line is given, so we can compare the worst-case complexity of the simple and advanced approach. With the simple approach, one local search iteration has complexity $O(pmk)$, because it costs $O(k)$ time to calculate the objective change of each of the $p \times m$ possible swaps. The advanced approach on the other hand, has an initialisation cost of $O(plk+mk)$ and each iteration takes $O(p(m+k)+lk+n)$. Since $O(plk+mk) \ll O(pmk+mk) \sim O(pmk)$, the initialisation costs are relatively small. The time complexity per iteration looks messier, but indicates a substantial improvement. If we split $O(p(m+k)+lk+n)$ in $O(p(m+k))+O(lk)+O(n)$, we can reason that if each part is much smaller than $O(pmk)$, then the sum of the three parts is also much smaller. So let us consider the separate parts: $O(p(m+k)) \ll O(pmk)$, because the product of m and k is many times larger than the sum. $O(lk) \ll O(pmk)$, as the sparsity of our data implies $l \ll m$ and the factor p is omitted. Finally, in our applications $O(n) \ll O(pmk)$, although in theory it is possible to find a test case for which this is not true. Summarizing, the worst-case complexity shows that the advanced approach can execute each local search iteration more efficiently than the simple approach. This is in line with what we expected after seeing the improved run-times of the advanced approach.

5.6 Path-Relinking

Implementing path-relinking is not difficult if the best improvement local search is already implemented. There are only two important differences between these techniques:

- Stopping criterion: In path-relinking, you always have to accept the best swap, even if it results in a negative profit. The process continues until the initial solution is identical to the guiding solution. Local search on the other hand, stops as soon as the best swap generates a profit smaller or equal to 0, as this means you have arrived at a local optimum.
- Candidate moves: In the best improvement local search, you can swap any active facility with any inactive facility. In path-relinking however, many moves are not allowed. A facility is only allowed to leave the initial solution if it does not occur in the guiding solution, and vice versa, the only candidates that are allowed to enter, are the facilities in the guiding solution. When the best of these candidate moves is executed, the initial solution moves one step towards the guiding solution, which implies even fewer possible swaps are left to consider.

In section 5.5, we discussed a simple and an advanced approach for the best improvement local search. This implies that there are also two approaches for the path-relinking process. Given the differences described above, it is straightforward to transform the simple best improvement algorithm into a path-relinking algorithm. This is more complicated for the advanced approach, so we will explain the necessary changes in more detail.

5.6.1 Path-relinking with the advanced approach

We will indicate which lines of the pseudo-code of Algorithm 4 need to be adjusted such that it can handle the differences between local search and path-relinking. See Algorithm 7 in the appendix for the the pseudo-code of the resulting path-relinking algorithm.

We start by initializing the candidate vector C with all facilities that are in the guiding solution, but not in the initial solution. We also introduce the vector with leaving facilities L, which are in the initial but not in the guiding solution. Lines 2 up to 18 of Algorithm 4 can be executed without changing anything, until we arrive at the stopping criterion in line 20. The path-relinking is finished when the initial solution is identical to the guiding solution,

so the stopping criterion consists of a test whether there are any elements left in L. Next, we calculate the profit matrix with a row for every $f_l \in L$ and a column for every $f_e \in C$, and determine which facilities correspond to the highest value in the profit matrix. The if-statement of line 24 is removed, because the best swap must always be executed. The swapped facilities cannot be used in future steps, so they are removed from L and C, which requires a small change in lines 26 and 27. Finally, we need to save the best solution found on the path connecting the initial and guiding solution, because this is not necessarily the last solution. After computing a new objective, we check whether it is higher than the best objective found so far, and if so, we save the current objective, solution and coverage vector. At the end of the algorithm, the properties of the best found solution are returned.

After applying these alterations to Algorithm 4, we already have a working path-relinking algorithm. We will now discuss some more adjustments that we can make to improve this implementation.

Note that a large part of the *extra* matrix is never used, because we only consider the rows and columns that correspond to facilities in L and C respectively. Constructing and adjusting the *extra* matrix takes a large portion of the total running time, so we can save time if we know which parts of *extra* are unnecessary. As we have seen in the previous section, *extra* and *loss* are filled row by row. We can check whether a row is relevant in a relatively quick way, so we have implemented this check in the algorithm. The columns of *extra* and the rows of *gain* are filled using the *col* dictionary. In order to know which of the facilities returned by the *col* dictionary are relevant, we would need to take the intersection with the candidate vector C. We found that taking this intersection is more expensive than simply filling all the columns indicated by *col*. Therefore, we adjusted the code such that only the relevant rows of *loss* and *extra* are constructed and updated, but this distinction is not made in the columns.

In line 6, we replace F^*_{idx} and F^* by L_{idx} and L, such that only the rows that correspond to facilities in L are constructed. We split the solution F^* in a constant part F^{con} and a variable part F^{var}. The variable part is initialized with all the elements of L, and in each path-relinking step, the entering facility replaces the leaving facility in F^{var}. If an improvement on the incumbent solution is found, we save the current composition of F_{var} and the corresponding objective and coverage vector. Splitting the solution avoids indexing problems due to the altered dimensions of *extra* and *loss*. At the end of the algorithm, the variable part corresponding to the best objective is concatenated to the constant, to obtain the full solution again.

In addition to restricting the construction, we also restrict the updating process. Updating *loss* and *extra* is done one row at a time, so per facility. Clearly, it is only necessary to update the rows which belong to the facilities in L, the ones that still have to leave the solution. We use the vector ϕ to make this distinction, this vector will indicate for every population point in *cov_21* and *cov_12* whether we need to process its associated change. Instead of initializing ϕ as a zero vector, we set every entry equal to -1. Next, for every facility f in L, we find the population points it uniquely covers, and fill their entries is ϕ with the index of f in F^{var}. In algorithm 4, $\phi(i)$ and $\phi_{old}(i)$ indicate which row of *extra* and *loss* need to be updated for every $i \in cov_21$ and $i \in cov_12$ respectively. Now, $\phi(i)$ fulfills the same function, but it is also possible that a -1 is returned. In that case, i is uniquely covered by a facility in the constant part of the solution, or by one of the

facilities that have entered the solution from C. Either way, that facility is not allowed to leave the solution, so we do not need to update the corresponding rows in *extra* and *loss*. Checking whether the value returned by ϕ is unequal to -1, suffices make sure we only update the relevant rows. Lastly, we delete lines 52, 55 and 60, in which the values corresponding to the leaving facility are replaced by the values of the entering facility. As the entered facility is not allowed to leave the solution in future steps, these lines are not necessary anymore.

This finalizes the explanation of the adaptations needed to convert the advanced best improvement local search algorithm into the advanced path-relinking algorithm. When comparing the run-times of the advanced and the simple approach, we observe that neither of the methods always outperforms the other one. In section 6.1.3 we will analyse when it is best to use which approach.

5.7 The full algorithm

So far we have discussed our implementation of each of the GRASP phases independently. The goal of this section is to demonstrate how the phases are connected, by providing an overview of the algorithm as a whole. This overview is given below in Algorithm 5, which is a description of all the steps of the GRASP algorithm, rather than a pseudo-code.

Algorithm 5: The full GRASP algorithm

Set p, S and *grid-size* to the desired values
Set a maximum running time t_{max}, and measure starting time *start*
Choose a local search strategy *best* or $first$, and if $first$ is chosen, set *border*
Choose a path-relinking strategy $forward$, *backward*, or *both*
Load the population data: $\mathcal{N}$, *pop*, coordinates of $\mathcal{N}$
Load the potential facility data: $\mathcal{M}$, coordinates of $\mathcal{M}$
Load the pre-processed dictionaries *row* and *col*
Adapt `Greedy` as described in 5.3.2 to add randomness
$F^*, cov, obj \leftarrow$ `Greedy(`$\mathcal{M}, \mathcal{N}, pop, row, col, p$`)`
if *best* **then**
  $F^*, cov, obj \leftarrow$ `Best_improvement_LS(`$F^*, cov, obj, \mathcal{M}, \mathcal{N}, pop, row, col, p$`)`
else
  $F^*, cov, obj \leftarrow$ `Local_Search(`$F^*, cov, obj, \mathcal{M}, pop, row, border$`)`
end
If F^* satisfies conditions section 3.4: store F^*, *cov* and *obj* in the elite pool
while $time - start < t_{max}$ **do**
  $F^*, cov, obj \leftarrow$ `Greedy(`$\mathcal{M}, \mathcal{N}, pop, row, col, p$`)`
  if *best* **then**
    $F^*, cov, obj \leftarrow$ `Best_improvement_LS(`$F^*, cov, obj, \mathcal{M}, \mathcal{N}, pop, row, col, p$`)`
  else
    $F^*, cov, obj \leftarrow$ `Local_Search(`$F^*, cov, obj, \mathcal{M}, pop, row, border$`)`
  end
  Randomly select an elite solution from the pool of elite solutions
  $forward$: set elite as guiding solution, *backward*: set elite as initial solution, *both*: perform $forward$ and *backward*, then pick the best outcome
  $F^*, cov, obj \leftarrow$ `PathRelink(`$F^*_{init}, cov_{init}, obj_{init}, F^*_{guide}, \mathcal{M}, \mathcal{N}, pop, row, col$`)`
  If F^* satisfies conditions section 3.4: store F^*, *cov* and *obj* in elite pool
end

6 Results Vietnam

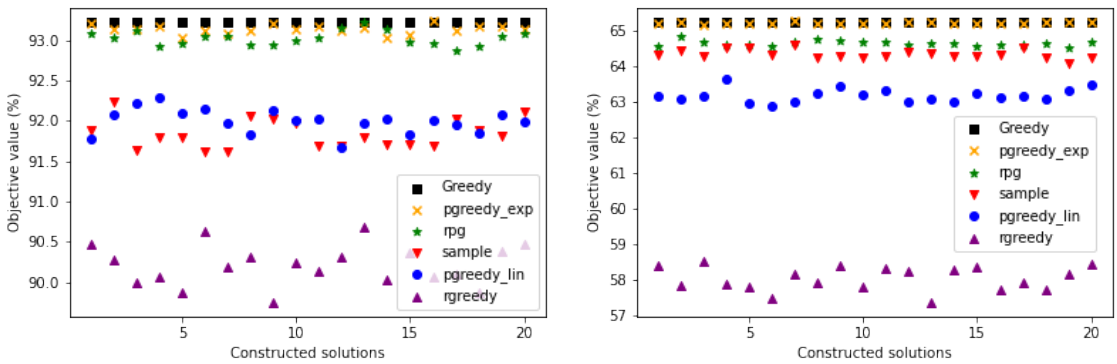
At this point, we have developed a 'toolbox' of algorithms, which can be combined in various ways in order to find a solution to a maximum covering problem. It is possible to pick the techniques and parameters which work best for a specific problem and time window, which makes this a generalizable solution method. Even though this is a strength of our product, it can be hard to determine which strategy choices lead to the best performance, as this is influenced by the problem size, the solution size and the available solving time.

If a solution is needed in a very short time window, it could be best to construct the solution in a completely greedy way, and perform a restricted first improvement local search on this greedy solution. This can give an indication of the solution value very quickly. However, if there is a longer time window, multiple GRASP iterations can be executed. In this case, a certain degree of randomness is introduced in the construction phase, and a choice for the local search and path-relinking strategy has to be made. In this section we will evaluate the performance of our algorithm in various test instances and using various KPI's. First, some results are reported on each of the GRASP phases independently, then the process as a whole is analysed in order to find the best combination of techniques and parameters for the Vietnam case. We end this chapter by finally applying these chosen settings to 1 km grid test instances.

6.1 Analysis of the separate phases

6.1.1 Construction phase

The quality and the amount of variation in the solutions that are obtained in the construction phase, depend on the method and parameter used to construct them. In Figure 3 the influence of the type of construction method on the objective value is visualised, for two distinct cases. Table 4 shows the corresponding maximum, average and standard deviation of the objectives, the average amount of overlap between two solutions, and the average time it took to construct the solutions. We used $\alpha = 0.05$ in $pgreedy_{exp}$, rpg, $pgreedy_{lin}$ and $rgreedy$, and $q = 0.02 * |\mathcal{M}|$ in *sample*. Note that *greedy* is simply the $rgreedy$ with $\alpha = 0$.



(a) Applied to $p = 60$, $S = 50$, *grid-size* $= 5$ (b) Applied to $p = 80$, $S = 20$, *grid-size* $= 1$

Figure 3: Comparing the objective values of constructed solutions

From these overviews, we can learn several things. The entirely *greedy* approach results

Table 4: Characteristics of constructed solutions

	$p = 60,\ S = 50,\ grid\text{-}size = 5$					$p = 80,\ S = 20,\ grid\text{-}size = 1$				
	$\max_{obj}$	μ_{obj}	σ_{obj}	$\mu_{\wedge}$	μ_{time}	$\max_{obj}$	μ_{obj}	σ_{obj}	$\mu_{\wedge}$	μ_{time}
greedy	93.24	93.24	0.00	60	1.1	65.25	65.25	0.00	80	5.5
$pgreedy_{exp}$	93.26	93.16	0.06	29	1.1	65.25	65.22	0.03	47	7.1
rpg	93.20	93.03	0.08	51	1.1	64.92	64.70	0.10	73	5.3
sample	92.28	91.92	0.19	3	0.4	64.59	64.33	0.09	3	6.9
$pgreedy_{lin}$	92.31	92.01	0.18	3	1.1	63.60	63.15	0.21	2	6.9
rgreedy	90.45	90.08	0.23	1	1.1	58.87	58.12	0.41	0	6.7

in the highest average objective in both cases, and naturally has no variation between its iterations. This makes *greedy* the best approach if the time limit only allows for one iteration. We observe that $pgreedy_{exp}$ closely follows the greedy solution, but with a slight variation. In both test instances, this small random factor even leads to finding a maximum objective that is higher than the greedy objective.

Looking at the rest of the methods, we observe that an increase in variation, measured by the standard deviation of the objective values, goes together with a decrease in the average objective value. Note that only measuring variation with the standard deviation can be a bit misleading. Consider solutions constructed with *rpg* for example, the randomly picked facilities create almost all of the variation, while the others are selected greedily. This means that there can be a lot of overlap between these solutions, even if their objective value varies substantially. This is indeed visible in Table 4, in which the column μ_{wedge} stands for the average overlap. The *rpg* solutions have more overlap than the $pgreedy_{exp}$ solutions, while judging from their standard deviation, we would expect that the variation in the *rpg* solutions would be larger.

Still, in general we see a clear trend that a larger standard deviation is often accompanied by a smaller overlap and a lower average objective value. This is in line with our expectations, because the objective is equal to the sum of all greedy values of the facilities that are added to the solution. Therefore, choosing a facility based on its greedy value is likely to be a better choice than picking a facility randomly. Only picking facilities with the highest greedy value is usually not the optimal choice of course, otherwise the greedy solution would already be optimal, but it is at least a fairly good choice. As it is not optimal, it is possible to find a better solution with one of the randomized greedy approaches, like we have seen for the $pgreedy_{exp}$ solutions. However, too much randomness means that often a facility with a relatively low greedy value will be selected, which will probably lead to a lower total as well. This phenomenon explains why *rgreedy* results in lower objectives than $pgreedy_{lin}$, even though they both pick from the top 5% facilities. *rgreedy* has no bias, whereas $pgreedy_{lin}$ is biased towards the options with the higher greedy values, which will often be a better choice.

The methods *sample* and $pgreedy_{lin}$ seem to have both a good amount of variation, and a fairly high quality. These are the most important requirements for the starting solutions according to the theory that we discussed in section 3.2, so we expect that these two methods should work well in combination with the rest of the GRASP algorithm. In section 6.2

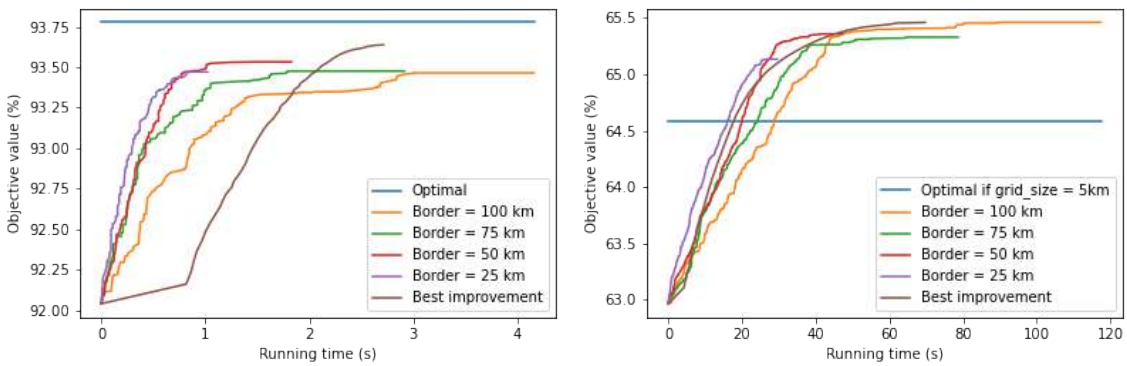
we perform an analysis of the complete GRASP algorithm, and we will examine if these methods are indeed the preferred methods.

As a final note, we would like to point out the difference in the time needed to compute the solutions. In the left table, the *sample* method is about 3 times faster than the other construction methods. The reason for this is that *sample* only needs to compute the greedy values for the facilities in the random sample, instead of for all facilities. This advantage vanishes if we look at the the right table. As we only update the greedy value of the affected facilities in our Greedy algorithm, using *sample* does not always lead to a time reduction. However, it is never slower, because in that case we can simply keep track of all greedy values, as in the Greedy algorithm, and pick the best one out of a new random sample at every step.

6.1.2 Local Search Phase

Next, we have the local search phase. In this phase, we have the choice between a first and best improvement strategy, and if the first improvement is chosen, we have to set a border. Recall that the value of *border* determines the size of the neighbourhood of each facility in the current solution. For example, if *border* = 50, then we will only try to swap each facility with potential facilities that are located closer than 50 kilometers from the considered facility. To get a feeling for the influence of the local search strategy on the quality of the local optima, and the time taken to find them, we have plotted several sample runs for the same cases that we discussed in the previous section.

Figure 9 shows the development of the best objective value during first improvement local search with borders 100, 75, 50 and 25 km, and during best improvement local search. Furthermore, the plots also show the optimal value obtained with Gurobi for $p = 60,\ S = 50$ and $p = 80,\ S = 20$, both computed using the 5 km grid. Each local search starts from the same solution, such that we can properly compare their development. This starting solution is generated with $pgreedy_{lin}$ and $\alpha = 0.05$, because this appeared to be a promising approach in the previous section. Note that a more thorough analysis of the performance of various construction methods in combination with the rest of the GRASP algorithm will follow in section 6.2.



(a) Applied to $p = 60$, $S = 50$, *grid-size* $= 5$ (b) Applied to $p = 80$, $S = 20$, *grid-size* $= 1$

Figure 4: Local search sample runs with varying strategies

The first thing that we notice, is that each graph easily exceeds the optimal value in Fig-

ure 4b. This illustrates the added value of solving the problem at a higher detail level. If more locations are considered, naturally the global optimum will increase, unless the optimal locations of the smaller problem would be exactly the optimal locations of the large problem, which is unlikely. And indeed, from the figure it is clear that the first local optimum we find when using the 1 km grid, is already better than the global optimum of the 5 km grid.

From the plot we can see the differences between the various strategies. In Figure 4a the initialisation cost of the best improvement strategy is clearly visible, it takes almost 1 second to fill the auxiliary structures *loss*, *extra* and *gain*, before the first swap is made. After this first swap, these structures only need to be updated, and the new best swaps are found a lot faster. In both graphs, the best improvement strategy finishes earlier than the first improvement search with 75 and 100 km as the *border* value, but later than the 25 and 50 km ones.

The influence of the border value on the local search curve is clearly visible. As the border increases, the neighbourhood of each facility increases, and it will usually take longer to complete. However, the resulting local optimum is not necessarily better, as you can see in Figure 4a, where $border = 50$ yields the highest objective value of all the first improvement strategies. From these figures it appears that a neighbourhood restricted at 25 km is too small, and that the 75 km and 100 km border take too long to complete. As it is too early to draw any conclusions based on these two figures, we will test three local search settings in the next section, to see how they function within the whole GRASP algorithm.

We would like to end by briefly highlighting the added value of implementing the advanced approach of best improvement local search. Due to this sophisticated implementation, we can finish a best improvement in a similar time as a first improvement local search, which is restricted quite severely in its allowed moves. It is now realistic to use the best improvement local search as a part of the GRASP heuristic, which would never have been possible with the simple approach. We have included Table 11 in the appendix, which shows how many times faster the advanced approach is compared to the simple approach. Both are applied to a random-greedy solution generated with $pgreedy_{lin}$ and $\alpha = 0.05$. We have only reported the results on the 10 and 5 km grid, because it took too long to execute the simple approach on the 1 km grid. From this table, we conclude that the advanced approach is up to 500 times faster than the simple approach. We expect that the acceleration factor is even larger for the 1 km grid, because the speed-up on the 5 km grid is also larger than on the 10 km grid. Clearly, the advanced approach results in a great run-time reduction of the best improvement local search across all considered test cases.

6.1.3 Path-Relinking

Once a local optimum is found, we will explore the path between this local optimum and one of the elite solutions via path-relinking. In section 5.6 we described a simple and an advanced approach to path-relinking, and noted that neither of these approaches is always faster than the other one. Similarly to what we have seen with the best improvement local search, the advanced approach has an initialisation cost for filling the auxiliary structures. As it turns out, the number of swaps that have to be considered in path-relinking is too small sometimes to compensate for this initialisation cost.

In the best improvement local search, you have to consider $p \times m - p$ swaps at every step, and because m is very large in our application, using the advanced approach is always faster than the simple implementation. This is completely different with path-relinking, where the maximum number of swaps to consider is $p \times p$ in the first iteration, $p-1 \times p-1$ in the second iteration, and so forth. If there are any facilities that occur in both the initial and guiding solution, the number of considered swaps decreases even further. While analysing the running time of the path-relinking procedure, we found that there is a clear relationship between the number of facilities that have to be switched, and the time taken by both methods. In Figures 5 and 6 we have plotted the number of facilities to switch on the y-axis, against the ratio of the run-time of the advanced approach over the run-time of the simple approach on the x-axis. The service distance and grid-size are the same as in the previous examples, but we let the p vary in order to analyse this relationship.

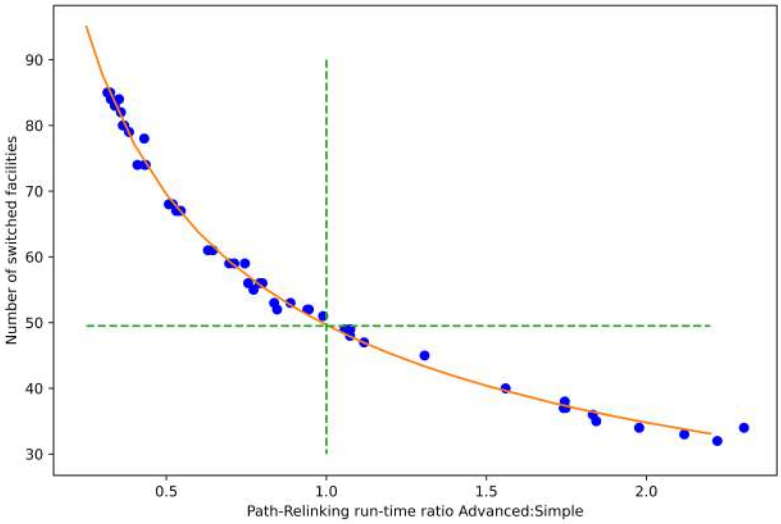


Figure 5: Run-time ratio path-relinking approaches on $S = 50$, *grid-size* $= 5$

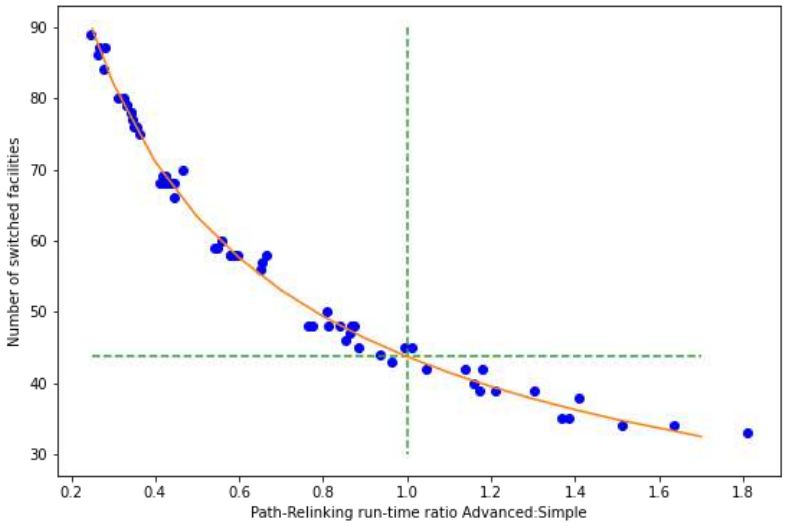


Figure 6: Run-time ratio path-relinking approaches on $S = 20$, *grid-size* $= 1$

In both figures, we observe that the advanced approach becomes more favourable as the number of different facilities in the initial and guiding solution increases. The curve

grid-size	S = 20	S = 50	S=100
10 km	35	50	38
5 km	45	49	43
1 km	43	96	84

Table 5: Turning points for using the advanced path-relinking implementation

seems to have two asymptotes, which can be explained by looking at the run-times of the approaches independently. Figures 11 and 12 in the appendix show that the advanced approach has a initialisation cost, but increases linearly with the number of switched facilities. The simple approach does not have this initialisation cost, but increases quadratically. Therefore the simple approach is much faster for a small number of switches, and a lot slower if more switches have to be executed. The asymptotes of the curve of the run-time ratio are a direct consequence of these observations.

Even though we see the same general relationship in both cases, the turning point and the steepness of the curve are also influenced by the grid-size and service distance. In figure 6, the turning point lies lower, and the time reduction achieved by the advanced approach is greater than in the first case.

We will use the fact that we have two methods to execute the path-relinking procedure to our advantage. We start with an analysis of each grid-size / service distance combination that we want to consider, in order to determine where the turning point lies. When executing the GRASP algorithm, we will count the number of facilities in the initial solution that are not in the guiding solution, and if this is larger than the predefined turning point, we will use the advanced approach. If not, we will use the simple implementation. Table 5 shows the turning points that we obtained for each case.

Finally, we would like to remark that we have included the choice for backward, forward, or back- and forward path-relinking in the analysis of the whole process, instead of in this section. Which of these strategies is optimal, depends on how successful the procedure is in finding new improving solutions, and how much time is costs in comparison to the other phases.

6.2 Analysis of the whole process

Now that we have examined each of the phases independently, we turn to our actual goal, which is combining the phases to an approach that can solve the Vietnam case at the highest detail level. Since it will take too long to test all of the different settings on this huge problem, we will tune our heuristic on the medium-sized case, which uses the 5 km grid to represent the potential facilities.

We compare the performance of the algorithm on several cases, with varying values of S, the service distance, and p, the number of new facilities. Which time limit we set is the next important decision. If we pick the best parameters assuming you can perform 100 GRASP iterations on the 5 km grid, but you only have time to finish 10 iterations on the 1 km grid, the chosen settings will probably not be optimal. So in an attempt to let our tests be representative of the performance on the 1 km grid, we set the time limit to 40 'standard GRASP iterations.' We define one standard GRASP iteration as constructing

a *rgreedy* solution, with $\alpha = 0.1$, followed by a best improvement local search, and then conducting forward path-relinking. We take the average time of 10 test iterations, and set the time limit to 40 times this average. We set a different time limit for each combination of grid size, service distance and number of new facilities.

We have chosen for 40 iterations, because most of the 1km grid cases can finish 40 standard iterations in less than 2 hours. For the other cases, we also cap the time limit at 2 hours. We think 2 hours to solve the biggest case is acceptable, considering that important investment decisions have to be made based on the outcome. Note that a 'quick and dirty' solution can be found a lot faster, but this is the limit we set for the high quality solution search. We will test various combinations of techniques and parameters using this same time limit. As a result, their advantages and disadvantages are automatically weighed out against each other. For example, if you set the sample size q small enough, the *sample* construction method will be faster than *rgreedy* or *pgreedy*, because you only evaluate the facilities in the random sample. Less time spent on constructing solutions means there is time for more iterations, or a more extensive path-relinking strategy, which can lead to finding better solutions. However, if q is too small, the solution quality will be low and a lot of time will be spent in the local search phase, which actually negates any time savings on the construction phase. Finding the right balance is key, and by testing various methods on the same case with the same time limit, we can observe which combination of techniques and parameters result in the best balance.

The set up of our experiment is as follows. The test cases that we consider have service distances $S = 20, 50, 100$ and $p = 20, 40, 60, 80, 100, 200$, using the 5 km grid to represent the potential facilities. For the service distance of 100 km, we excluded any p over 80, because the optimal objective does not increase anymore. This means we have 16 test cases, and we apply all possible combinations of construction methods and local search strategies on each of these cases. For each of the construction methods *rgreedy*, *rpg*, $pgreedy_{lin}$, $pgreedy_{exp}$, and *sample*, we have 4 values for the randomness parameter. For *sample* we take $q \in \{0.01 * |\mathcal{M}|, 0.02 * |\mathcal{M}|, 0.05 * |\mathcal{M}|, 0.10 * |\mathcal{M}|\}$, and for the other methods we test $\alpha \in \{0.01, 0.05, 0.10, 0.20\}$. Each of these configurations is tested three times, each time with a different local search strategy. We either perform the first improvement local search with $border \in \{50, 100\}$, or the best improvement local search.

Finally, we had to make a decision on the path-relinking strategy. We did not want to test each of the mentioned solving configurations with forward, backward, and back- and forward path-relinking, as that would make it even harder to interpret the results. The preliminary tests that we ran, showed that path-relinking only took a small amount of the total run-time, even more so because we can always pick the most efficient implementation. Furthermore, back- and foward path-relinking seemed to increase the number of improvements found compared to choosing one of the two approaches. Thus we chose to always execute the procedure in both directions. We kept track of the proportion of improvements found by each of the directions, so we can analyse whether they both contribute to finding better objectives.

We run each of the discussed settings on each of the cases, using the time limits given in Table 12 in the appendix. In each run, we keep track of several KPI's: the best found objective, the best found objective at half of the time limit, and two time-to-target values. We define a target at 0.05% below the optimum, which we consider a negligible difference.

	Best	Halftime	TTT	TTT2
1st	***sam*,0.02,*best*:58**	$sam, 0.05, best : 50$	***sam*,0.02,*best*: 38**	$rpg, 0.05, best : 42$
2nd	$lin, 0.2, best : 58$	***sam*,0.02,*best*: 49**	$sam, 0.05, best : 34$	$rpg, 0.1, best : 40$
3rd	$lin, 0.01, best : 54$	$lin, 0.01, best : 49$	$exp, 0.2, best : 23$	$exp, 0.1, best : 39$
4th	$sam, 0.05, best : 53$	$lin, 0.1, best : 47$	$rpg, 0.1, best : 22$	***sam*,0.02,*best*: 38**
5th	$lin, 0.1, best : 52$	$sam, 0.1, best : 45$	$sam, 0.1, best : 21$	$exp, 0.01, best, 36$

Table 6: Comparison construction phase strategies, using best improvement in local search phase

	Best	Halftime	TTT	TTT2
1st	$sam, 0.05, 50 : 58$	***sam*,0.02,50: 49**	***sam*,0.02,50: 35**	***sam*,0.02,50: 45**
2nd	$sam, 0.01, 50 : 55$	$sam, 0.05, 100 : 49$	$exp, 0.01, 50 : 25$	$exp, 0.05, 50 : 37$
3rd	***sam*,0.02,50: 53**	$sam, 0.1, 50 : 46$	$sam, 0.05, 50 : 25$	$rpg, 0.01, 50 : 36$
4th	$sam, 0.1, 50 : 53$	$sam, 0.01, 50 : 45$	$sam, 0.1, 50 : 17$	$exp, 0.1, 50 : 35$
5th	$sam, 0.05, 100 : 53$	$sam, 0.02, 100 : 42$	$sam, 0.05, 100 : 17$	$exp, 0.01, 50 : 32$

Table 7: Comparison construction strategies, using first improvement in local search phase

The second target we set at 0.2 % below the optimum, which we consider an acceptable difference. We call these KPI's TTT and TTT2 respectively.

Once everything is executed, we compare the approaches using these KPI's. We first split the part of the obtained data that uses best improvement from the part which uses first improvement, because we think that the local search strategy might influence which construction strategy yields the best results. Then, for each of the cases, we determine the top 5 scores on each KPI. If a construction method has the best score, it gets 5 points, the second best 4 points, and so forth. Then we sum these scores over all the cases to obtain a total amount for each construction method. Table 6 shows an overview of the methods with the 5 best total scores for each KPI. In Table 7 the result of the same analysis for the test runs with a first improvement local search is given. The names in the table indicate which settings were used. *sam* stands for the *sample* construction method, *lin* for $pgreedy_{lin}$ and *exp* for $pgreedy_{exp}$. The type of construction method is followed by the value of the randomness parameter. Next, the local search strategy is indicated by *best* or the value of the *border* in the first improvement case. Finally the total score is also given, which makes it possible to assess how big the performance differences are between the methods.

In the first table, *sample* and $pgreedy_{lin}$ are the methods that occur most frequently, with varying values for the randomness parameters q and α. $pgreedy_{lin}$ does well with its score on a high final and halftime objective, but apparently does not reach these values very quickly, as it does not occur in the top five of TTT or TTT2. *rpg* seems to work well to get to an acceptable solution very quickly, but does belong to the top methods when it comes to the final solution quality. *sample* is the only construction method that occurs in the top five of all four KPI's, and even has the highest score on three of them. Two of the three top scores of *sample* are obtained with $\alpha = 0.02$, and this setting also occurs in the top five of the other 2 KPI's.

	Best	**Halftime**	**TTT**	**TTT2**
1st	$sam, 0.02, best : 59$	$sam, 0.02, 50 : 55$	$sam, 0.02, best : 47$	$sam, 0.02, 50 : 47$
2nd	$sam, 0.05, 50 : 57$	$sam, 0.05, best : 53$	$sam, 0.05, best : 40$	$sam, 0.02, best : 40$
3rd	$sam, 0.1, 50 : 56$	$sam, 0.1, 50 : 50$	$sam, 0.02, 50 : 32$	$sam, 0.05, 50 : 37$
4th	$sam, 0.02, 50 : 55$	$sam, 0.05, 100 : 49$	$sam, 0.1, best : 30$	$sam, 0.05, best : 37$
5th	$sam, 0.05, best : 53$	$sam, 0.02, best : 47$	$sam, 0.05, 50 : 22$	$sam, 0.1, 50 : 27$

Table 8: Comparison construction parameters and local search strategies, using *sample* construction

Based on this table, we conclude *sample* is the best construction method, and $\alpha = 0.02$ is the best parameter setting. This construction method might not be the best method for every individual case, but we prefer to have a method that works well for a variety of test instances. We do want to remark that we should not blindly follow the outcomes of this table, because it is just one way of analysing the performance. However, *sample* also emerged as a promising approach from our analysis of the construction phase on its own, so we set it as our preferred method to use in combination with the best improvement strategy.

Following a similar reasoning for while examining the results in Table 7, we again find *sample* with α to be the method with the highest potential. Focusing solely on the *sample* construction mechanisms, we combine the first and best improvement data, to analyse which method works best overall. Table 8 shows the corresponding outcomes. We observe that all the values seem to be quite close to each other, but there are some slight differences. $\alpha = 0.02$ yields the best results, it occurs twice in each top five, combined with both best improvement and restricted first improvement with $border = 50km$. The latter scores better on the halftime objective and time to the acceptable target, whereas the best improvement has a better score on the other two KPI's. Based on this table, we decide to use the best improvement strategy as long as the 40 standard GRASP iterations can be executed in under 2 hours. If we have to cap the solving time, we will use the first improvement with $border = 50$, because in this setting the algorithm seems to converge faster.

Finally we would like to report some results on the contribution of path-relinking in the solving process. We found that 98% of all improving solutions found across all cases and methods was found during the path-relinking procedure. Of these improving solutions, 29% were found during forward path-relinking, and the other 71% during backward path-relinking. This is in line with our expectations, based on the related literature discussed in section 3.4. Backward path-relinking seems to work better than forward path-relinking, and back- and forward path-relinking in turn seems to improve on solely backward path-relinking. The obvious disadvantage of performing this procedure twice, is that it also takes twice the time. The proportion of the total available time spent on path-relinking ranges from <0.01% up to 15% in some cases. This percentage is usually relatively high when the first improvement search is restricted at 50 km, because relatively little time is spent in the local search phase in those cases. Overall, we think it is acceptable to spend this amount of time to execute the path-relinking procedure, because it is such an important tool in finding new best solutions.

6.3 Outcomes of the tuned algorithm on the largest case

Everything that we have discussed so far serves the goal of solving a huge maximal covering problem. Specifically, we want to be able to optimally allocate the stroke facilities in Vietnam, while considering every square kilometer of Vietnam as a potential facility location. Now that we have found ways to cope with all speed and memory challenges encountered while developing this heuristic, and after tuning the approach such that we expect the best performance on the 1 km grid, we are keen to share these results with the reader.

		grid-size = 5km		**grid-size = 1km**		**Time (s)**		
S	p	Opt	GRASP_5	GR+BI	GRASP_1	Opt	GR+BI	GRASP_1
20	20	48.868	48.868	49.447	49.484	11.3	8	42.7
	40	55.464	55.464	56.224	56.319	12.2	10.4	44.4
	60	60.483	60.483	61.325	61.452	15.7	17.2	769.3
	80	64.581	64.581	65.495	65.670	17.9	26.2	3099.5
	100	68.127	68.125	69.208	69.275	19.5	69.9	577.1
	200	80.038	80.013	81.137	81.396	32.6	274.1	6644.6
50	20	83.574	83.574	83.833	83.848	106.7	24.3	256
	40	90.137	90.137	90.353	90.469	33.9	40.1	2112.8
	60	93.777	93.773	93.938	94.064	177.7	65.2	4062.5
	80	96.091	96.091	96.196	96.377	157.3	96.1	4768.5
	100	97.550	97.541	97.592	97.743	436.8	162	3902.8
	200	99.896	99.877	99.876	99.913	96.1	367.4	7054.5
100	20	98.266	98.266	98.220	98.303	266.7	75.9	3280.5
	40	99.565	99.556	99.521	99.569	288.2	105.4	3440.3
	60	99.891	99.882	99.886	99.892	179.0	131.3	2743.1
	80	99.972	99.969	99.978	99.983	50.2	125.2	2169.8

Table 9: Optimal values compared to GRASP outcomes and to quick approximations

Table 9 summarizes the most important results regarding the final outcomes of the developed heuristic. For the 5 km grid, the optimal values and the GRASP outcomes are given. We used Gurobi to find the optimal solution, with an allowed MIP-GAP of 0.01 %. GRASP was run with $sample, \alpha = 0.02$ in the construction phase, combined with a best improvement local search, and back- and forward path-relinking. In each iteration, a choice for which path-relinking approach is used is made, based on the number of facilities that have to be swapped. GRASP uses the lime limits given in Table 12 in the appendix. We observe that the GRASP outcomes on the 5 km grid are almost identical to the optimal values found by Gurobi: the largest difference is only 0.025%.

Another way of assessing the difference between the GRASP outcome and the optimal solution, is by plotting the chosen facilities on the map of Vietnam. Figure 7 and 8 demonstrate the outcomes for the case with $S = 50$, *grid-size* $= 5$ and $p = 60$ in the north and south of Vietnam. The blue points are existing facilities, the orange points indicate locations that occur both in the optimal solution and in the heuristic solution, the green points are only chosen by the heuristic solution, and finally the red ones are only in the optimal solution. We can see the accessibility of stroke facilities for people living

in rural areas has increased substantially, whereas no extra facilities have been allocated in the areas where there were already a lot of facilities. In the North, almost all of the facility locations chosen by the heuristic, are also an optimal choice. In the South, there are some more small differences. We observe that as soon as one facility is not the same in the heuristic and optimal, there are a few facilities that are just located differently. For example, at the bottom of figure 8, there are 4 heuristics facilities located, and 3 optimal ones. Due to the big influence of local search, which optimizes by swapping 1 facility at a time, you cannot escape this local optimum, because removing one of the 4 facilities in the area, would probably result in a worse objective.

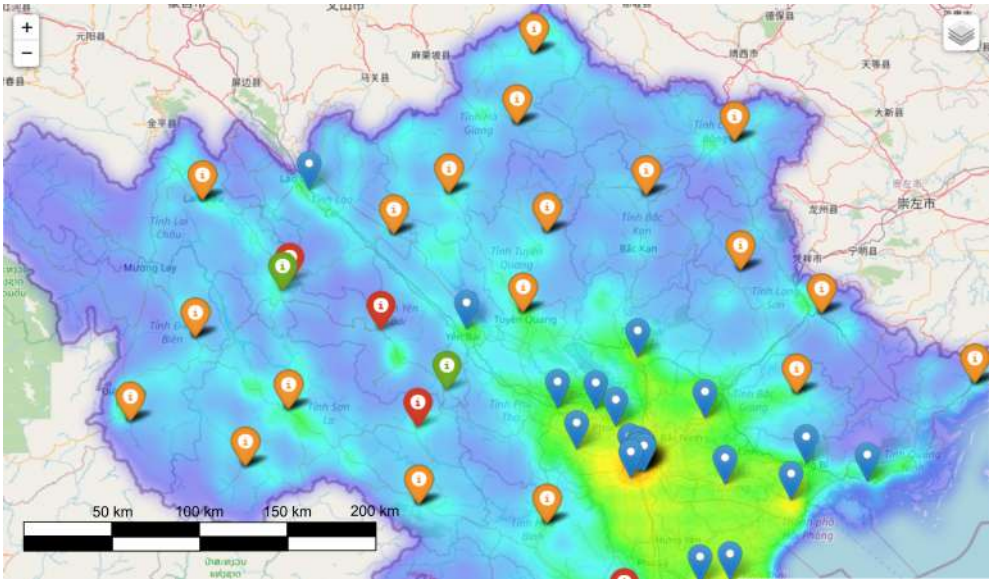


Figure 7: Heuristic and optimal outcomes of stroke facility allocation, North Vietnam

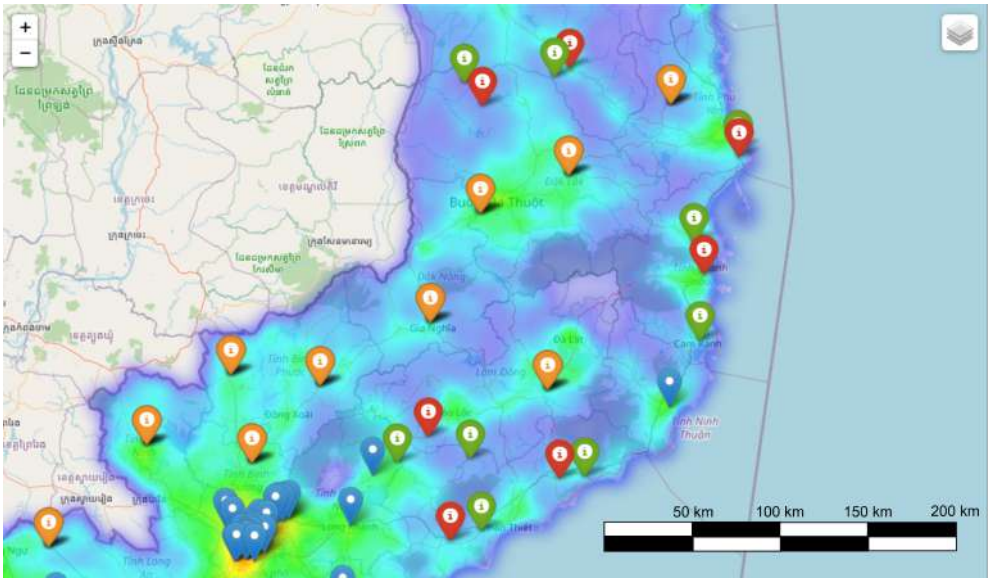


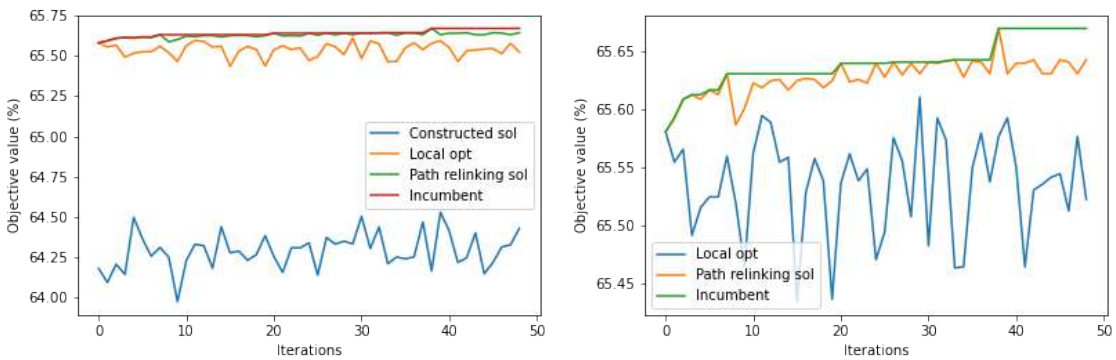
Figure 8: Heuristic and optimal outcomes of stroke facility allocation, South Vietnam

If we move our attention to the results belonging to the results on the 1 km grid in Table

9, the advantage of using a 1 km grid is immediately clear. We report the results of an approximation approach GR+BI, which stands for greedy plus best improvement. This means we constructed one completely greedy solution, and applied the best improvement local search to it. For 12 of the 16 cases, the outcome of this quick approach applied to the 1km grid is already higher than the optimal value in the 5 km grid. In the Time columns 'Opt' and 'GR+BI', we also compare the time taken by Gurobi to find the optimum, and the time taken by the approximate approach to find a solution. Sometimes our the approximation method is much faster than the exact optimizer, and sometimes the opposite is true. Our method becomes relatively slow when p increases, because the local search takes a lot longer to complete. In these case, using the restricted first improvement strategy could be a good alternative.

There are two columns that we have not discussed yet, the $GRASP_1$ columns. These contain the objective values which are found by applying the full GRASP algorithm to the 1km grid, and the time taken to reach this objective. For each case, the GRASP outcome for the 1 km grid is higher than the optimum of the 5 km grid. Especially for the instances with covering distance 20, the increase is substantial. If $p = 80$, 100 or 200, more than 1% of the Vietnamese inhabitants can be covered extra, as a direct result of using more detailed data. If we take into account that around 97 million people live in Vietnam, it is definitely worthwhile to solve the problem at this level and to spend a maximum of 2 hours to run each instance.

We would like to finish this discussion of our results by showing a sample run of the 1 km grid case that we have seen several times in this section. We show the development of the objectives found in the construction, local search, and path-relinking phase, as the iterations progress. In the figure to the right, we have omitted the construction phase objective, so the path of the local search, path relinking and best objective is clearer.



(a) Applied to $p = 80$, $S = 20$, $grid\text{-}size = 1$ (b) Applied to $p = 80$, $S = 20$, $grid\text{-}size = 1$

Figure 9: Objective development of each phase during GRASP iterations

We observe that the objective values resulting from the construction phase have some variation, but on average the data quality remains constant as the algorithm progresses. This is in line with our expectations, because the construction method has a random factor, and is independent of the rest of the algorithm. The outcomes of the best improvement local search show a substantial increase in objective compared to the constructed solution, and there is still quite a bit of variation in the outcomes of the local search. This is a good sign, because this implies that we do not end up in the same local optimum every

time, but also explore new regions of the solution space. Finally, the added value of the path-relinking process is evident, as every solution that improves the incumbent solution is found in the path-relinking phase. Furthermore, we observe that there is a slight increasing trend in the path-relinking objectives, which is probably due to an increase in the average solution quality of the pool of elite solutions.

7 Performance compared to the literature

In this section we will apply our heuristic to a test instance solved in recent literature. This gives us the chance to test if the heuristic generalizes well to new data. If this is the case, there will be more future applications for which Analytics for a Better World will be able to use this heuristic. The second reason for this comparison is that we can examine whether the choices that we have made yield better results than the methods previously published. The most recent paper with the largest maximal covering problem solved in the literature is written by Máximo et al. [15] in 2019, in which they develop a GRASP based heuristic. This paper is similar to a paper from 2017 of the same writers [16], the main difference being the addition of path-relinking as an intensification strategy. Both papers focus on an algorithm called 'Intelligent Guided Adaptive Search', or IGAS, which uses an artificial neural network to influence the solutions that are constructed in subsequent iterations. The performance of IGAS is then compared to a standard GRASP heuristic, and in the 2019 paper, both algorithms are extended with path-relinking and are compared again.

As the most important requirement for our heuristic is scalability, we have chosen to exclude the dependency on an artificial neural network, and keep our approach more transparent. Therefore, we have not been able to reproduce the IGAS results reported by Máximo et al., but we did reconstruct their GRASP algorithm. The results reported in the paper however, differ noticeably from the results that we found after carefully replicating their described approach. We must mention that the papers failed to mention many details regarding the implementation. For example, nothing is written on the way an objective value corresponding to a solution is calculated, or whether they use a sparse or a dense matrix to represent the data.

We will report both the results reported by the paper, and the results that we found in our reconstruction of their method. We will then compare these with the results obtained by our own heuristic, using the settings that we found to work the best in section 6. Note that the essence of both approaches is the same: both are GRASP heuristics and make use of path-relinking. The approaches differ on the following points:

- Our method uses the construction method *sample*, with $q = 0.02 * \mathcal{M}$, as opposed to a *rgreedy* approach with $\alpha = 0.15$.
- In the local search phase, we use the best improvement local search described in section 5.5. The paper of Máximo et al. simply refers to the first improvement local search method implemented by Resende in 1998 [18]. This strategy continues from the same place in the loops after a swap has been made, instead of returning to the 'base'. In Algorithm 2, this would mean returning to line 4 instead of line 2.
- Our method has a choice between using the simple path-relinking implementation, or the advanced one. After running some preliminary tests, we conclude that if the number of facilities that have to be swapped exceeds 25, it is usually faster to use the advanced approach. So in every iteration we compute the difference between the guiding and initial solution, and choose the path-relinking implementation accordingly.
- We use forward and backward path-relinking, whereas the approach described in the paper only executes one of the two at every iteration, starting from the solution with the highest objective.

Size: 7730 x 7730		**Approach Máximo et al.**				**Approach thesis**			
p	**LP values**	Best	Worst	Med	Time	Best	Worst	Med	Time
50	2,033,678.47	1.150	2.143	1.624	227.6	0.594	1.104	0.737	236.9
60	2,255,707.14	2.135	2.786	2.379	223.9	1.551	1.953	1.717	260.0
70	2,410,847.41	2.364	3.130	2.796	206.4	1.717	2.390	2.069	277.8
80	2,490,711.39	2.070	2.661	2.297	207.0	1.527	2.025	1.800	280.0
90	2,504,700.00	0.707	1.105	0.930	219.4	0.417	0.713	0.582	284.6
100	2,504,700.00	0.117	0.320	0.236	238.4	0.010	0.079	0.040	279.7

Table 10: Comparison thesis heuristic with best available literature alternative

Now that we have established the differences between the two algorithms, we present the table with the results in Table 10. The test instance consists of 7730 demand nodes, and each of the demand nodes functions as a potential facility location. The distances are calculated using the spherical law of cosines, and the covering distance is 800 m. For every value of p, the algorithm is executed 30 times, each time with a time limit of 300 seconds. The quality of a solution is assessed using the gap between its objective value and the linear relaxation value of the corresponding test instance. For test instance, the best, worst and median gap, and the average time to get to the best solution are reported. For improved readability, we do not show the results reported by Máximo et al here, but only the values obtained by our reconstruction. The results reported in their paper can be found in Table 13 in the appendix.

We can be fairly brief in our analysis of the results in the table. The heuristic that was developed in this thesis outperforms the reconstructed approach of Máximo et al. accross all instances. Each of the best, worst and median gap is smaller if we use our approach, but the average time needed to reach the best solution is a bit higher. Even though our heuristic is mainly designed for solving much larger instances than this one, this experiment shows that it can also be applied to smaller problems.

As a final comment, we would like to add that the outcomes of our approach are not better than the ones reported in the paper of Máximo et al, which can be found in Table 13 in the appendix. Our algorithm finds better solutions for $p = 90$ and $p = 100$, and their reported outcomes are better for the other values of p.

8 Conclusion

The aim of this thesis is to develop a heuristic which can solve very large scale maximal covering problems, and specifically the problem of allocating new stroke facilities in Vietnam. The novelty of our final product is twofold: the size of the solved problem vastly exceeds the size of any other maximal covering problem solved in the literature, and the best improvement local search strategy developed by Resende and Werneck [23] is adapted to a maximal covering setting for the first time.

Many challenges, either speed or memory related, arise when attempting to solve a maximal covering problem of approximately 350,000 potential facility locations, by 300,000 demand points. A binary matrix of such dimensions is too large for the average workstation, calculating road distances between all of these points may take an exorbitant amount of time, and commercial solvers get a memory error when trying to solve the linear relaxation. The size of test instances in the literature on solving large maximal covering problem only goes up to 8,000 by 8,000, while using the haversine distance as the distance metric. To put it mildly, it seemed ambitious to solve a problem of this order of magnitude, but still, we set out to create a GRASP heuristic, because it seemed to be the most promising solution method for large MCLPs in the literature.

A sparse matrix implementation is used to overcome the memory issues, and, if implemented in a smart way, it contributes to increasing the speed of the algorithm as well. The combination of using a coverage vector to represent a solution, and only accessing its relevant entries by using the sparse matrix as an indexing tool, allows us to efficiently calculate the objective corresponding to a solution, and of small solution changes. This technique runs like a thread through all parts of the algorithm we developed, and is applied in many different ways. In addition to increasing the efficiency of calculations, we also aim to minimize the number of redundant calculations. For example, we restrict the neighbourhood of each facility in first improvement local search, or we use additional data structures to store computations which are needed in multiple iterations.

The latter is done in the greedy algorithm, the greedy value of each facility is saved in a vector, and is only adjusted if it is affected by the facility entering the solution. We noticed a resemblance with Resende and Werneck's [23] implementation of the best improvement local search on a p-median problem. The difference was that their approach could handle the addition and removal of a facility at the same time, which makes it a very efficient method to execute swap-based algorithms, such as local search. We decided to translate their approach to a maximum covering setting, such that we could apply it in the local search phase of our GRASP heuristic.

To the best of our knowledge, this translation has never been made in the literature so far. Implementing this advanced technique was found to be very useful, in our experiments it is up to 500 times faster than the simple implementation of best improvement local search. We also applied these ideas to create an advanced approach of the path-relinking algorithm. The run-time of the simple implementation increases quadratically with the number of steps from the initial to the guiding solution, whereas the advanced approach increases linearly, but has an initialisation cost. This implies that the advanced approach results in a substantial acceleration as soon as the number of steps is larger than some threshold. Finally, our translation of this technique to the maximum covering setting, also

facilitates applying it to other covering problems, such as the set cover problem.

Once all the GRASP phases had been implemented in an efficient way, we set up a grading system to evaluate the performance of the algorithm while using a variety of settings. The GRASP configuration that emerged as the most successful from this experiment, was then used to solve the Vietnamese stroke facility problem at the highest detail level of 1 km. The time allowed for running the GRASP algorithm varies per test instance, but never exceeds 2 hours. For each case, the GRASP outcomes improved upon the optimal value of the problem at the lower detail level of 5 km, with coverage increases of up to 1.4 percentage points. The resulting objective is equivalent to the optimal value on the 5 km grid, when 65 instead of 60 facilities are added. This demonstrates the added value of being able to solve maximum covering problems at a higher detail level.

As an alternative to the full GRASP algorithm, the approximation approach can be used to quickly give an indication of the objective value of a certain case. This approach consists of computing a fully greedy solution, and applying a best improvement local search to it. When testing this approach on the 1 km grid, it already found better objectives than the 5km optimum in 12 of the 16 cases. Furthermore, the approximations are all within 0.3% of the best found GRASP outcomes, so indeed they give a good indication of the result that can be obtained by the full process. The approximation could be useful as an initial analysis, such that a decision maker can assess which coverage values he can expect for various values of p and S. Based on this estimation, a few promising values can be selected for which the full GRASP algorithm will be executed.

We conclude that we have fulfilled the main goal of this thesis. We are able to solve each test instance of the Vietnam stroke facility allocation problem in less than 2 hours on a standard 16 GB RAM computer, so the heuristic is time and memory efficient. It performs well on a wide range of values for S and p, as well as on a completely different data set from the literature, so it is also generalizable. Finally, the request for a 'quick and dirty' solution method is satisfied with the approximation approach, which only takes a few minutes to run.

The theoretical model used to represent the stroke facility allocation problem, also has its limitations however. As of yet, we are not able to include capacity constraints in this model, which might be unrealistic in a health care environment. Moreover, the covering distance has a great influence on the obtained solution. More research is necessary on what travel distance is acceptable, taking into account the time that is lost before someone recognizes the symptoms of a stroke, transportation is arranged, and the patient is admitted to the hospital. Data-wise it would be valuable to have access to travel times instead of travel distances, or at least to more complete road network data. Demographic data could be used to estimate the stroke risk of people living in certain area, so the demand vector could be adjusted accordingly.

Another interesting research direction is extending or improving the GRASP heuristic itself. It might be valuable to incorporate knowledge obtained in earlier iterations in the construction phase already, such that the quality of constructed solutions increases as the algorithm progresses. This might lead to higher quality solutions or faster convergence. A second idea is to implement a self-tuning construction method, that automatically adjusts the randomness parameter to a value that yields the best results.

References

[1] Susan Armijo-Olivo, Arianna Waye, and Lisa Tjosvold. *Endovascular Therapy for Acute Ischemic Stroke*, chapter 2: Social and System Demographics. Edmonton (AB): Institute of Health Economics, 2017.

[2] Pasquale Avella, Maurizio Boccia, Saverio Salerno, and Igor Vasilyev. An aggregation heuristic for large scale p-median problem. *Computers & Operations Research*, 39(7):1625–1632, 2012.

[3] Jenny F. Calderín, Antonio D. Masegosa, and David A. Pelta. An algorithm portfolio for the dynamic maximal covering location problem. *Memetic Computing*, 9(2):141–151, 2017.

[4] Richard Church and Charles ReVelle. The maximal covering location problem. In *Papers of the regional science association*, volume 32, pages 101–118. Springer-Verlag, 1974.

[5] Jean-François Cordeau, Fabio Furini, and Ivana Ljubić. Benders decomposition for very large scale partial set covering and maximal covering location problems. *European Journal of Operational Research*, 275(3):882–896, 2019.

[6] Mark S. Daskin. What you should know about location modeling. *Naval Research Logistics (NRL)*, 55(4):283–294, 2008.

[7] Institute for Health Metrics and Evaluation (IHME). Global burden of disease study 2019 (GBD 2019) Compare. `https://vizhub.healthdata.org/gbd-compare/`, 2020. Accessed: 2021-07-12.

[8] Roberto D. Galvão, Luis G.A. Espejo, and Brian Boffey. A comparison of lagrangean and surrogate relaxations for the maximal covering location problem. *European Journal of Operational Research*, 124(2):377–389, 2000.

[9] Roberto D. Galvão and Charles ReVelle. A lagrangean heuristic for the maximal covering location problem. *European Journal of Operational Research*, 88(1):114–123, 1996.

[10] Michael R. Garey and David S. Johnson. *Computers and intractability: a guide to the theory of np-completeness*, volume 174. Freeman San Francisco, 1979.

[11] Edward L Hillsman. The p-median structure as a unified linear model for location—allocation analysis. *Environment and Planning A*, 16(3):305–318, 1984.

[12] Chandra A. Irawan, Said Salhi, and Maria P. Scaparra. An adaptive multiphase approach for large unconditional and conditional p-median problems. *European Journal of Operational Research*, 237(2):590–605, 2014.

[13] Manuel Laguna and Rafael Marti. Grasp and path relinking for 2-layer straight line crossing minimization. *INFORMS Journal on Computing*, 11(1):44–52, 1999.

[14] Luiz A.N. Lorena and Marcos A. Pereira. A lagrangean/surrogate heuristic for the maximal covering location problem using hillman's edition. *International Journal of Industrial Engineering*, 9:57–67, 2002.

[15] Vinícius R. Máximo and Mariá C.V. Nascimento. Intensification, learning and diversification in a hybrid metaheuristic: an efficient unification. *Journal of Heuristics*, 25(4):539–564, 2019.

[16] Vinícius R. Máximo, Mariá C.V. Nascimento, and André C.P.L.F. Carvalho. Intelligent-guided adaptive search for the maximum covering location problem. *Computers & Operations Research*, 78:129–137, 2017.

[17] Marcos A. Pereira, Luiz A.N. Lorena, and Edson L.F. Senne. A column generation approach for the maximal covering location problem. *International Transactions in Operational Research*, 14(4):349–364, 2007.

[18] Mauricio G.C. Resende. Computing approximate solutions of the maximum covering problem with grasp. *Journal of Heuristics*, 4(2):161–177, 1998.

[19] Mauricio GC Resende and Celso C Ribeiro. A grasp with path-relinking for private virtual circuit routing. *Networks: an international journal*, 41(2):104–114, 2003.

[20] Mauricio G.C. Resende and Celso C. Ribeiro. Greedy randomized adaptive search procedures: Advances, hybridizations, and applications. In *Handbook of metaheuristics*, pages 283–319. Springer, 2010.

[21] Mauricio G.C. Resende and Celso C. Ribeiro. Greedy randomized adaptive search procedures: Advances and extensions. In *Handbook of metaheuristics*, pages 169–220. Springer, 2019.

[22] Mauricio G.C. Resende and Renato F. Werneck. A hybrid heuristic for the p-median problem. *Journal of heuristics*, 10(1):59–88, 2004.

[23] Mauricio GC Resende and Renato F Werneck. A fast swap-based local search procedure for location problems. *Annals of Operations Research*, 150(1):205–230, 2007.

[24] Charles ReVelle, Michelle Scholssberg, and Justin Williams. Solving the maximal covering location problem with heuristic concentration. *Computers & Operations Research*, 35(2):427–435, 2008.

[25] Celso C. Ribeiro and Isabel Rosseti. Efficient parallel cooperative implementations of grasp heuristics. *Parallel Computing*, 33(1):21–35, 2007.

[26] Francisco J. Rodriguez, Christian Blum, Manuel Lozano, and Carlos García-Martínez. Iterated greedy algorithms for the maximal covering location problem. In *European Conference on Evolutionary Computation in Combinatorial Optimization*, pages 172–181. Springer, 2012.

[27] Marilyn M. Rymer, Naveed Akhtar, Coleman Martin, and Debbie Summers. Management of acute ischemic stroke: time is brain. *Missouri medicine*, 107(5):333, 2010.

[28] Edson L.F. Senne, Marcos A. Pereira, and Luiz A.N. Lorena. A decomposition heuristic for the maximal covering location problem. *Advances in Operations Research*, 2010, 2010.

[29] Hillary Ta, Bryant Lin, and Latha Palaniappan. Vietnamese and vietnamese-american health statistics, 2003-2019. *CARE Data Brief, Stanford Center for Asian Health Research and Education*, 2, 2020.

[30] Michael B. Teitz and Polly Bart. Heuristic methods for estimating the generalized vertex median of a weighted graph. *Operations research*, 16(5):955–961, 1968.

[31] RA Whitaker. A fast algorithm for the greedy interchange for large-scale clustering and median location problems. *INFOR: Information Systems and Operational Research*, 21(2):95–108, 1983.

[32] Mohammad H.F. Zarandi, Soheil Davari, and Seyyed A.H. Sisakht. The large scale maximal covering location problem. *Scientia Iranica*, 18(6):1564–1570, 2011.

A Appendices

Existing stroke facilities in Vietnam

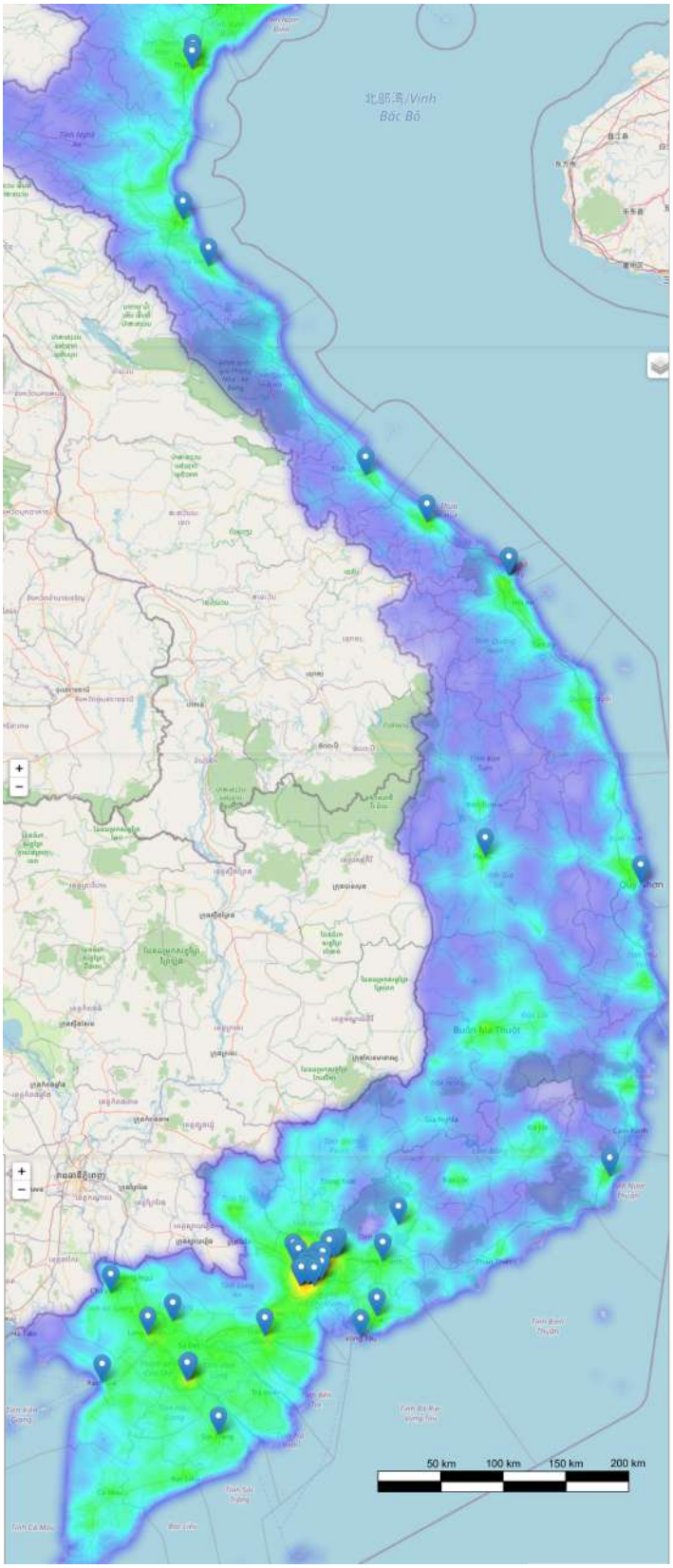


Figure 10: Existing stroke facilities in the south of Vietnam

Advanced Path-relinking Algorithm

Algorithm 6: Path-Relinking Function

Function `Path_Relinking`$(F^*, cov, obj, F^*_{guide}, \mathcal{M}, \mathcal{N}, pop, row, col)$
  Initialize the vector of candidate facilities $C \leftarrow \{F^*_{guide}\} \backslash \{F^*\}$
  Initialize the vector of leaving facilities $L \leftarrow \{F^*\} \backslash \{F^*_{guide}\}$
  Split F^* in a constant and variable part: $F^{con} \leftarrow \{F^*\} \cap \{F^*_{guide}\}$, $F^{var} \leftarrow L$
  Set p equal to the length of L
  Set $obj_{best} \leftarrow obj$, $cov_{best} \leftarrow cov$, $F^{var}_{best} \leftarrow F^{var}$
  Initialize $loss$ and $gain$ as zero vectors of length p and $|\mathcal{M}|$ respectively
  Initialize ϕ as a vector with all entries equal to -1 and length $|\mathcal{N}|$
  Initialize $extra$ as zero matrix of size $p \times |\mathcal{M}|$
  Initialize L_{idx} as an index vector of length p with values $0, \ldots, p-1$
  for $f_{idx} \in L_{idx}$ and $f \in L$ **do**
    $\mathcal{OUT} \leftarrow \{i \in row[f] : cov(i) = 1\}$
    $loss(f_{idx}) \leftarrow \sum_{i \in \mathcal{OUT}} pop(i)$
    $\phi(i) \leftarrow f_{idx}\ \forall i \in \mathcal{OUT}$
    for $i \in \mathcal{OUT}$ **do**
      $extra\Big(f_{idx}, col[i]\Big) \leftarrow extra\Big(f_{idx}, col[i]\Big) + pop(i)$
    end
  end
  for $f \in C$ **do**
    $\mathcal{IN} \leftarrow \{i \in row[f] : cov(i) = 0\}$
    $gain(f) \leftarrow \sum_{i \in \mathcal{IN}} pop(i)$
  end
  while $L \neq \emptyset$ **do**
    Execute vectorized profit function to obtain profit matrix:
     $profit(f_l, f_e) \leftarrow gain(f_e) - loss(f_l) + extra(f_l, f_e)\ \forall f_l \in L_{idx}, \forall f_e \in C$
    Let $f^*_{l,idx}$, $f^*_{e,idx}$ be the coordinates of the highest value in $profit$
    Let $f^*_l \leftarrow L(f^*_{l,idx})$ and $f^*_e \leftarrow C(f^*_{e,idx})$ be the corresponding facilities
    Delete entered facility from candidate list: $C \leftarrow \{C\} \backslash \{f^*_e\}$
    Similarly, update L and L_{idx}: $L \leftarrow \{L\} \backslash \{f^*_l\}$, $L_{idx} \leftarrow \{L_{idx}\} \backslash \{L_{idx}(f^*_{l,idx})\}$
     Replace f^*_l by f^*_e in F^{var}
    Update obj: $obj \leftarrow obj + profit(f^*_{l,idx}, f^*_{e,idx})$
    Increase coverage cov: $cov(i) \leftarrow cov(i) + 1\ \ \forall i \in row[f^*_e]$
    Decrease coverage cov: $cov(i) \leftarrow cov(i) - 1\ \ \forall i \in row[f^*_l]$
    if $obj > obj_{best}$ **then**
      Set $obj_{best} \leftarrow obj$, $cov_{best} \leftarrow cov$, $F^{var}_{best} \leftarrow F^{var}$
    end
    $\phi_{old} \leftarrow \phi$
    Reset ϕ to a vector with all entries equal to -1
    for $f_{idx} \in L_{idx}$ and $f \in L$ **do**
      $\mathcal{OUT} \leftarrow \{i \in row[f] : cov(i) = 1\}$
      $\phi(i) \leftarrow f_{idx}\ \forall i \in \mathcal{OUT}$
    end
    $overlap \leftarrow \{row[f^*_e]\} \cap \{row[f^*_l]\}$

```
    cov_21 ← {i ∈ row[f_l^*] : [cov(i) = 1] ∧ [i ∉ overlap]}
    for i ∈ cov_21 do
        if φ(i) ≠ −1 then
            loss(φ(i)) ← loss(φ(i)) + pop(i)
            extra(φ(i), col[i]) ← extra(φ(i), col[i]) + pop(i)
        end
    end
    cov_12 = {i ∈ row[f_e^*] : [cov(i) = 2] ∧ [i ∉ overlap]}
    for i ∈ cov_12 do
        if φ_old(i) ≠ −1 then
            loss(φ_old(i)) ← loss(φ_old(i)) − pop(i)
            extra(φ_old(i), col[i]) ← extra(φ_old(i), col[i]) − pop(i)
        end
    end
    cov_10 ← {i ∈ row[f_l^*] : [cov(i) = 0]}
    for i ∈ cov_10 do
        gain(col[i]) ← gain(col[i]) + pop(i)
    end
    cov_01 ← {i ∈ row[f_e^*] : [cov(i) = 1] ∧ [i ∉ overlap]}
    for i ∈ cov_01 do
        gain(col[i]) ← gain(col[i]) − pop(i)
    end
end
F* = F^con ∪ F_best^var
return F*, cov_best, obj_best
end
```

Acceleration factor of advanced approach

S	p	10 km	5 km
20	20	124.9	173.5
	40	149.8	239.1
	60	173.6	286.5
	80	285.5	280.9
	100	246.3	320.8
	200	334.4	401.5
50	20	43.5	99.1
	40	81.1	184.7
	60	100.5	245.3
	80	133.3	295.9
	100	175.1	342.7
	200	317.9	481.9
100	20	28.3	68.9
	40	60.6	159.3
	60	103.1	263.3
	80	184.4	315.3

Table 11: Acceleration factor of advanced vs simple approach on best improvement local search

Path-relinking run-times

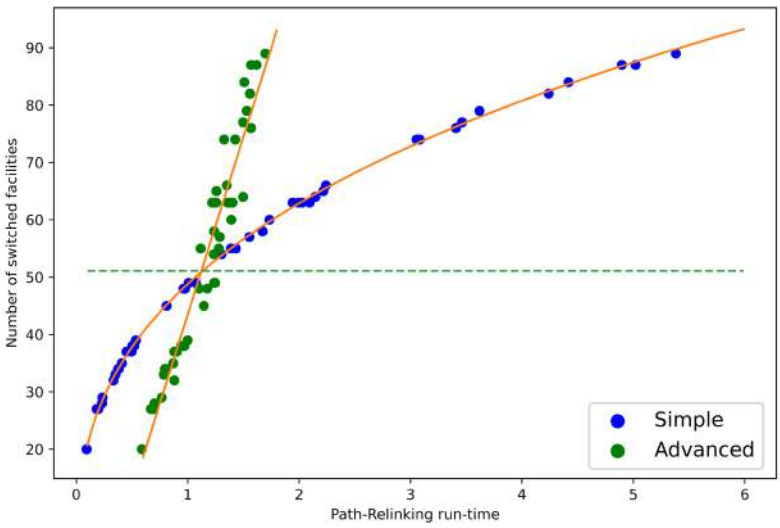


Figure 11: Run-times path-relinking, simple and advanced approach on $p = 60$, $S = 50$, *grid-size* $= 5$

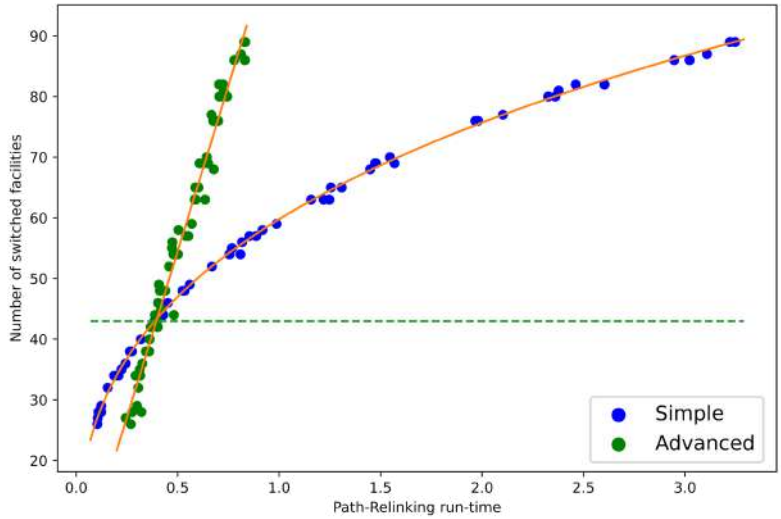


Figure 12: Run-times path-relinking, simple and advanced approach on $p = 80$, $S = 20$, $grid\text{-}size = 1$

Run-times Vietnam case

	$grid\text{-}size = 5km$			$grid\text{-}size = 1km$		
p	$S = 20$	$S = 50$	$S = 100$	$S = 20$	$S = 50$	$S = 100$
20	22.6	67.9	195.1	462.5	1093.7	3561.8
40	40.5	117.5	253.0	1090.8	2350.4	5012.2
60	68.5	180.8	284.7	2171.1	4153.6	6123.4
80	107.5	249.3	325.0	3837.2	5901.1	7200.0
100	149.5	316.5		6485.3	7200.0	
200	486.1	584.3		7200.0	7200.0	

Table 12: Time limits

Additional details Literature case

Size: 7730 x 7730		**Reported Results**				**Reconstructed Results**			
p	**LP values**	Best	Worst	Med	Time	Best	Worst	Med	Time
50	2,033,678.47	0.584	1.016	0.705	135.8	1.150	2.143	1.624	227.6
60	2,255,707.14	1.440	1.903	1.630	142.1	2.135	2.786	2.379	223.9
70	2,410,847.41	1.387	2.270	1.877	143.0	2.364	3.130	2.796	206.4
80	2,490,711.39	1.221	1.771	1.589	204.3	2.070	2.661	2.297	207.0
90	2,504,700.00	0.678	0.902	0.768	159.0	0.707	1.105	0.930	219.4
100	2,504,700.00	0.112	0.195	0.149	114.1	0.117	0.320	0.236	238.4

Table 13: Comparison between the reported and the reconstructed results